

게임핵 제작 실습

공부할 내용

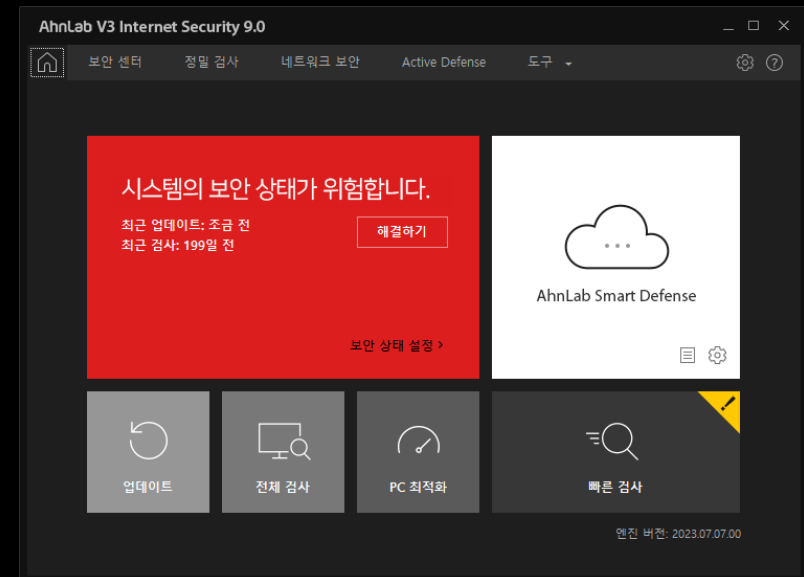
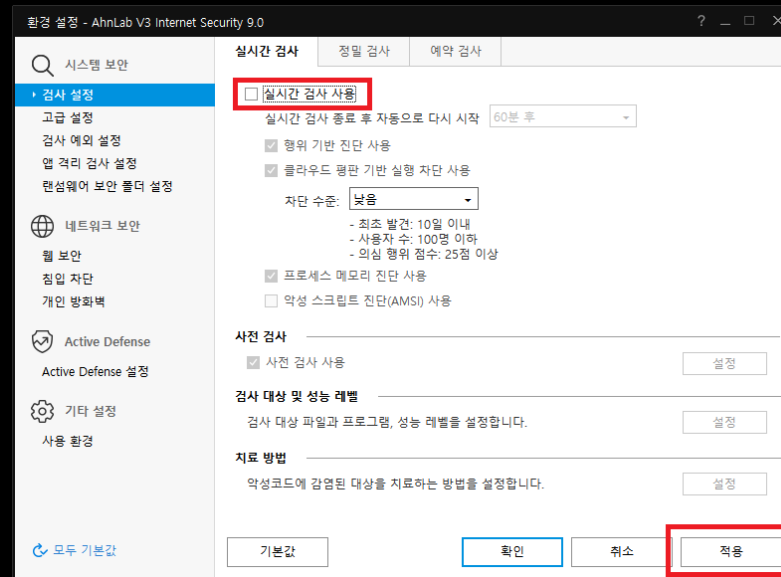
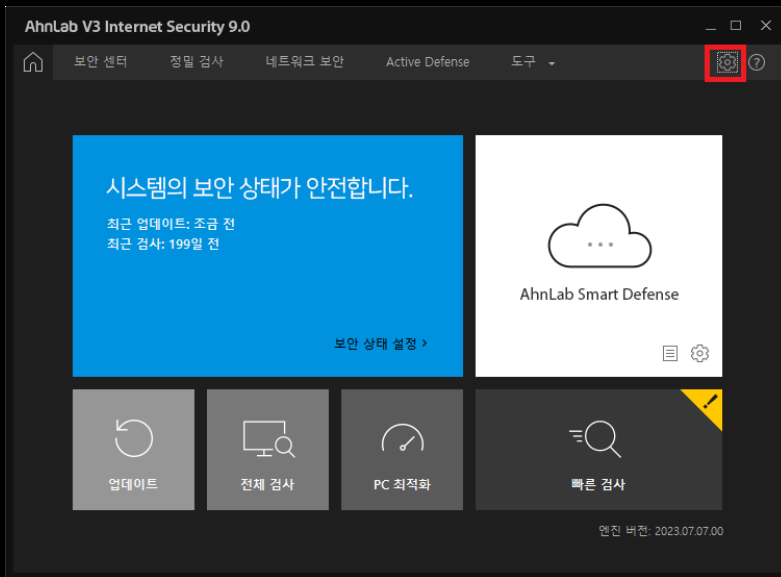
- 게임 핵을 제작해본다.

관련 분야

- Pwnable(시스템 해킹)
- Reverse Engineering(역공학)
- Web Hacking(웹 해킹)
- MISC(암호학)
- Digital Forensic(포렌식)

들어가기전..

- 백신을 잠시 꺼야 합니다.



사전 준비

- 게임 다운로드

- 피카츄 배구 : pika.exe
- Assault Cude : AssaultCube_v1.3.0.2_LockdownEdition.exe

- 치트 엔진 다운로드

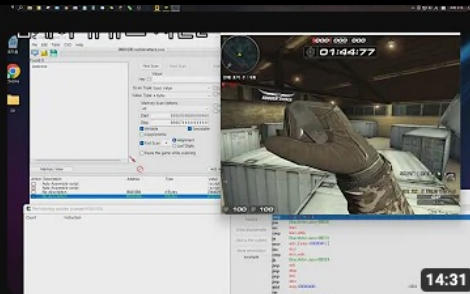
- https://www.cheatengine.org/#google_vignette

- x64dbg

- https://github.com/x64dbg/x64dbg/releases/download/snapshot/snapshot_2023-07-05_21-19.zip

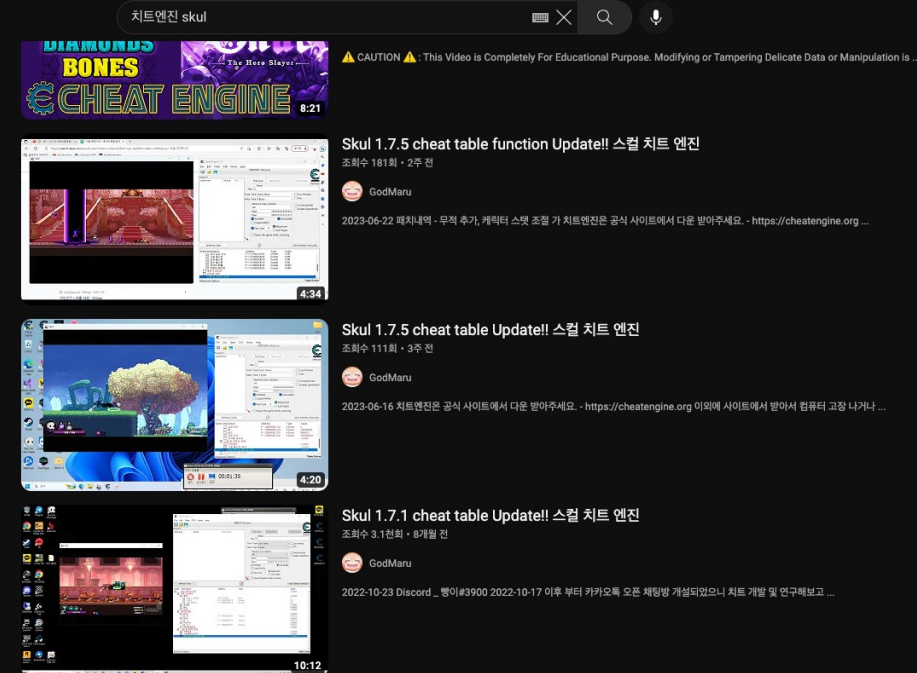
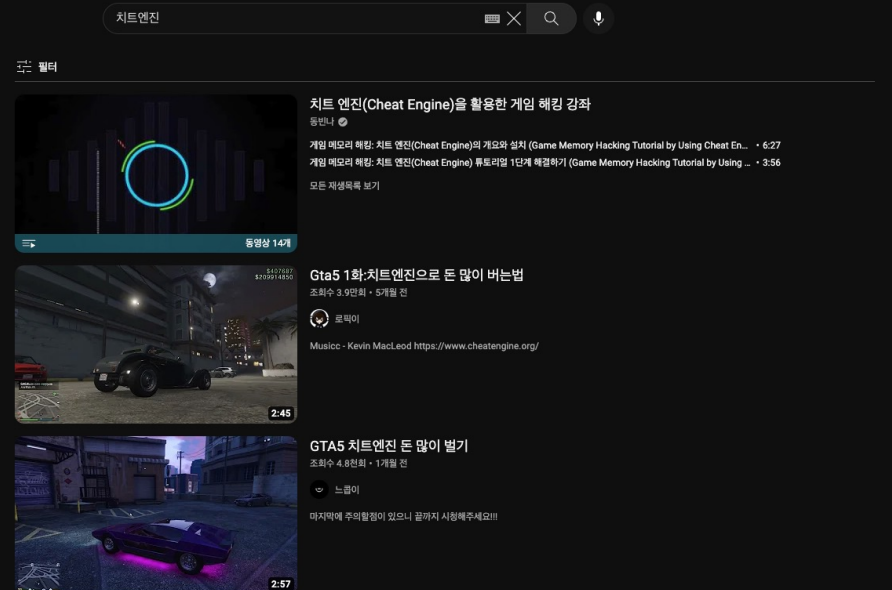
Cheat Engine

- 치트 엔진(Cheat Engine)은 **시스템 메모리 내의 데이터를 수정할 수 있도록 도와주는 소프트웨어입니다.**
- 굉장히 오래됐지만 지금도 꾸준히 사용되는 SW로 **동적 분석 툴**



Undetected Cheat Engine(치트엔진 7.4) 완벽 우회하는 방법. | NGS 2023 Bypass - All Games. |

조회수 7천회 · 4개월 전



x64dbg

- 기계어 코드부터 어셈블리어 코드를 생성해주는 SW
- 주로 동적 분석에 사용
- ollydbg 최신화 버전이라 생각하면 됨
- 동적 분석 도구

1. 치트 엔진 다운로드

백신을 꺼야만 다운로드가 된다.

[Download Cheat Engine](#)

able installing/running Cheat Engine? Check out the [known github issue](#) here on how to solve it, or join the [cheat engine patreon](#)

before download: You must be 18 years or older, or deemed an adult, to install Cheat Engine. Cheat engine is for private and educational purposes only. Before you attach Cheat Engine to a process, please ensure that you are not violating the EULA/TOS of the specific game/application. cheatengine.org does not condone the illegal use of Cheat Engine

February 14 2023:Cheat Engine 7.5 Released for public:

February 8 2023:Cheat Engine 7.5 Released for Windows for [Patreons](#) (Mac and public will be here soon):

glad to announce the release of Cheat Engine 7.5 which has some really neat improvements over 7.4
patreon members can get it [here](#)

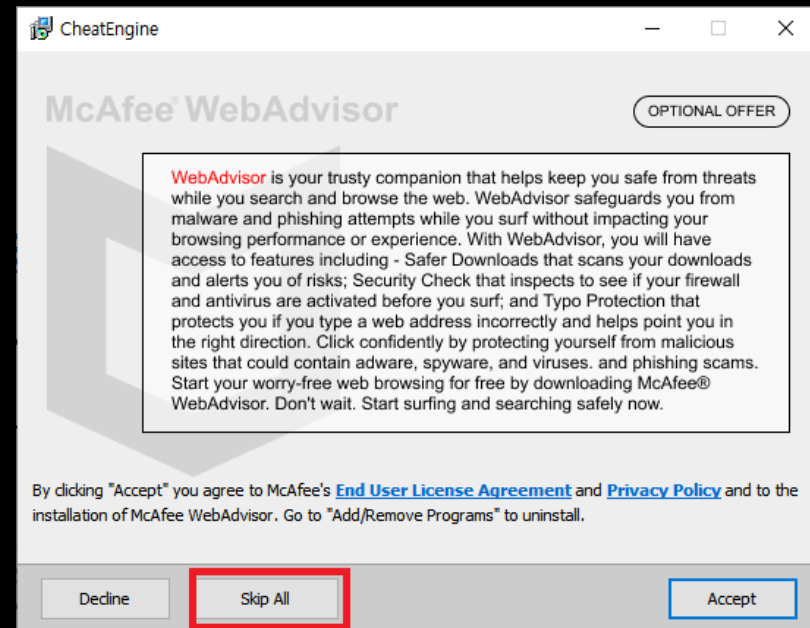
public release will be here any day now. Waiting for the advertisers / network owners to accept it)

se reports bugs and give suggestions to improve Cheat Engine so next version can be even greater!

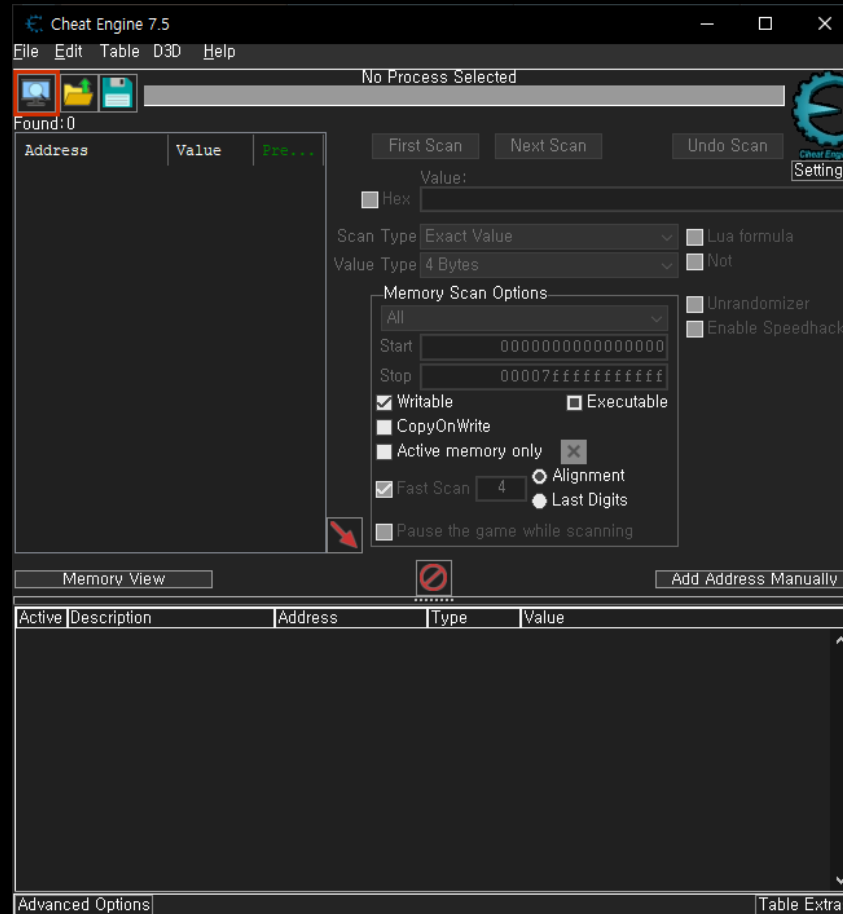
anges:

patreon 7.4.3 to public 7.5:

removed the driver requirement for the access memory regions tool
added 1 byte jmp instructions (that will install an exception handler and place an int3 at the location)
added a scanoption so you can skip unpaged memory. (should prevent targets from eating up RAM when scanned)
disassemble() now rewrites an instruction using multiple lines when needed
make some error messages more descriptive
added an option to center the highlighted disassembler code to the center
added an explanation why the driver won't load and a link with info on how to get it to load for now
memoryrecord hotkeys can now be disabled individually
codefilter: unwind info now gives less bad results

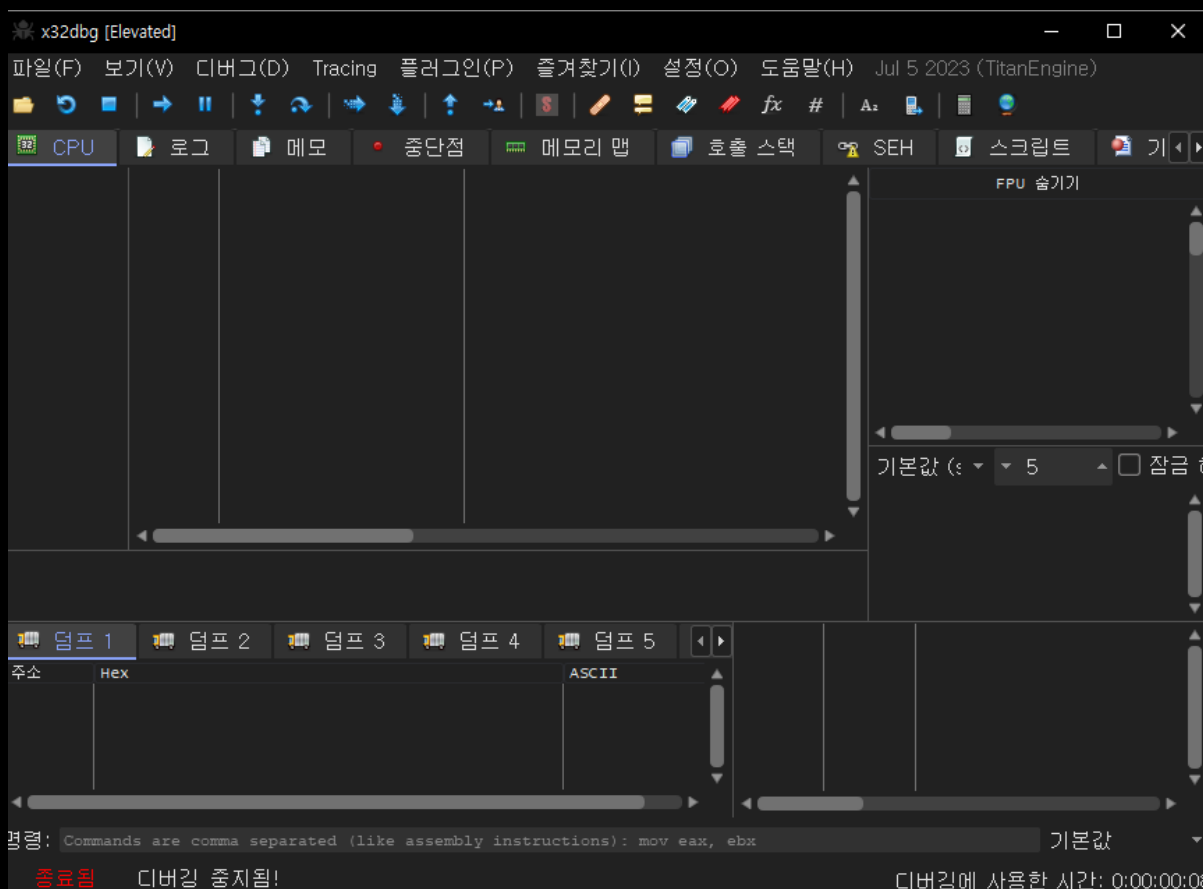


치트엔진 초기화면



2. x64dbg 다운로드

- snapshot_2023-07-05_21-19WreleaseWx32Wx32dbg.exe 파일을 열었을 때의 초기화면



개념 - 어셈블리어

- 어셈블리를 단기간에 공부하는건 언뜻 이해가 잘 가지 않는다.
- 실습을 진행하기에 앞서 몇가지 명령어만 알아보도록 하자.
- `mov eax, ebx` - 값 저장
- `inc` - +1
- `Jle` - jump less equal
- `jg` - jump greater

피카츄 배구를 해보자.



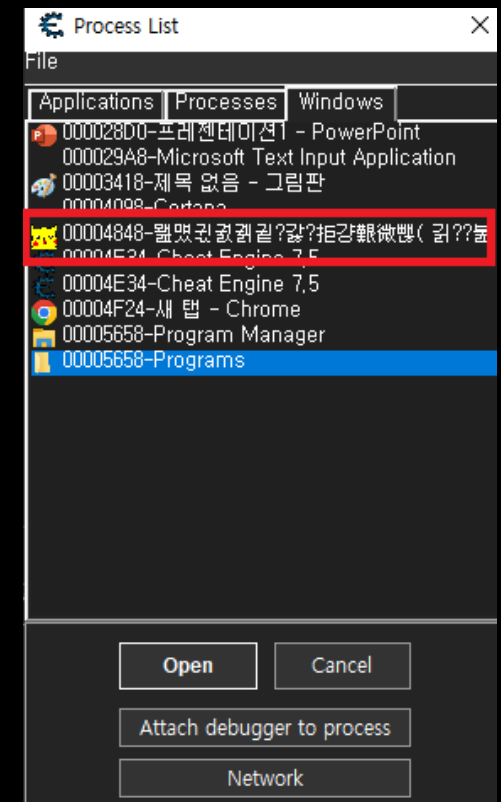
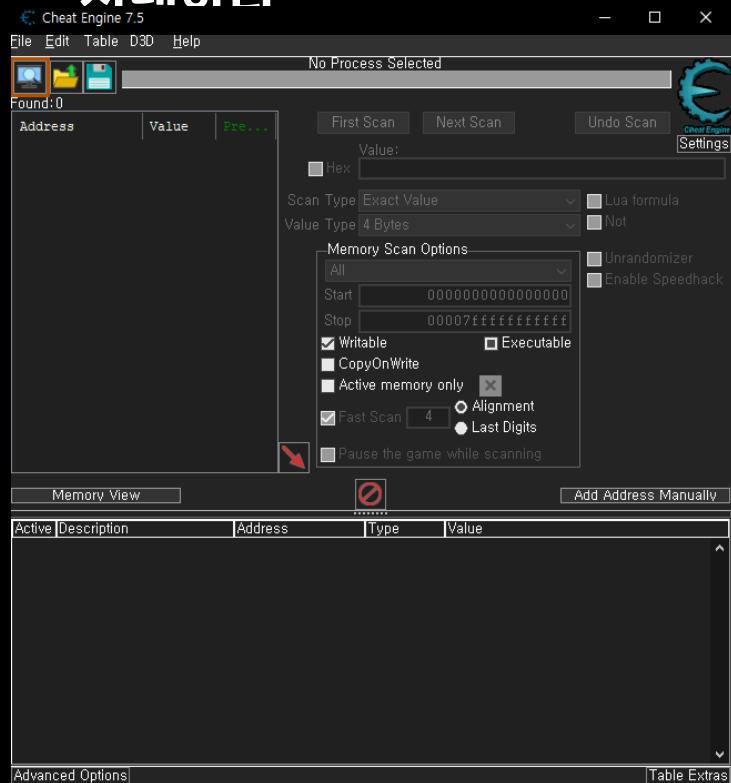
피카츄 배구를 하며 느낀점

- 컴퓨터가 은근 잘한다 ㅠㅠ

컴퓨터를 이길 수 있는 방법은?

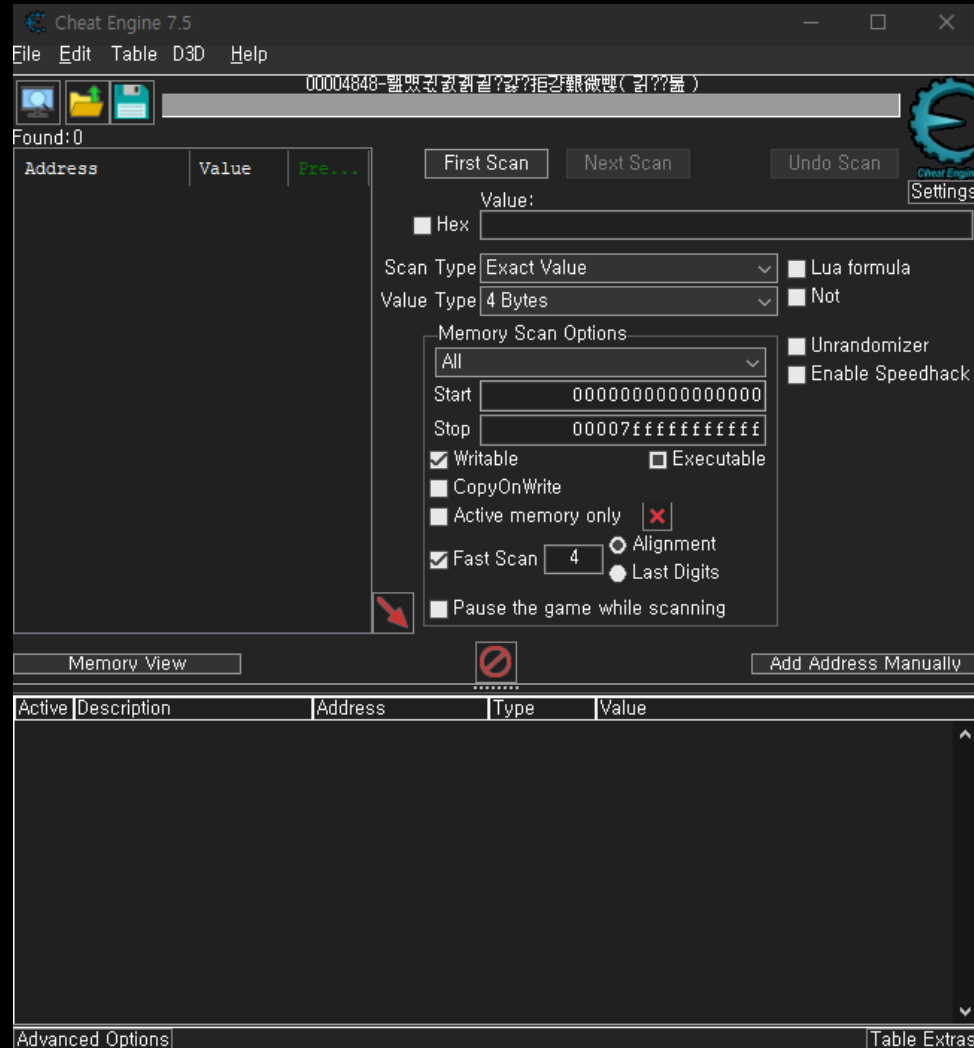
컴퓨터를 이겨보자...!

- 피카츄 배구 게임이 켜진 상태로 치트엔진 좌측 상단의 모니터를 클릭 후 피카츄 배구를 서태하다



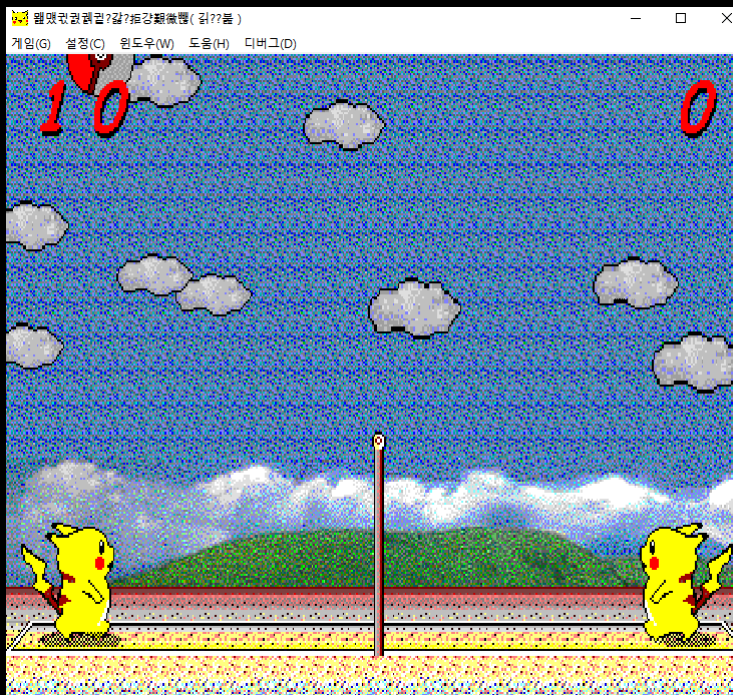
(C) 1997 SACHISOFT / SAWAYAKAN Programmers

컴퓨터를 이겨보자...!

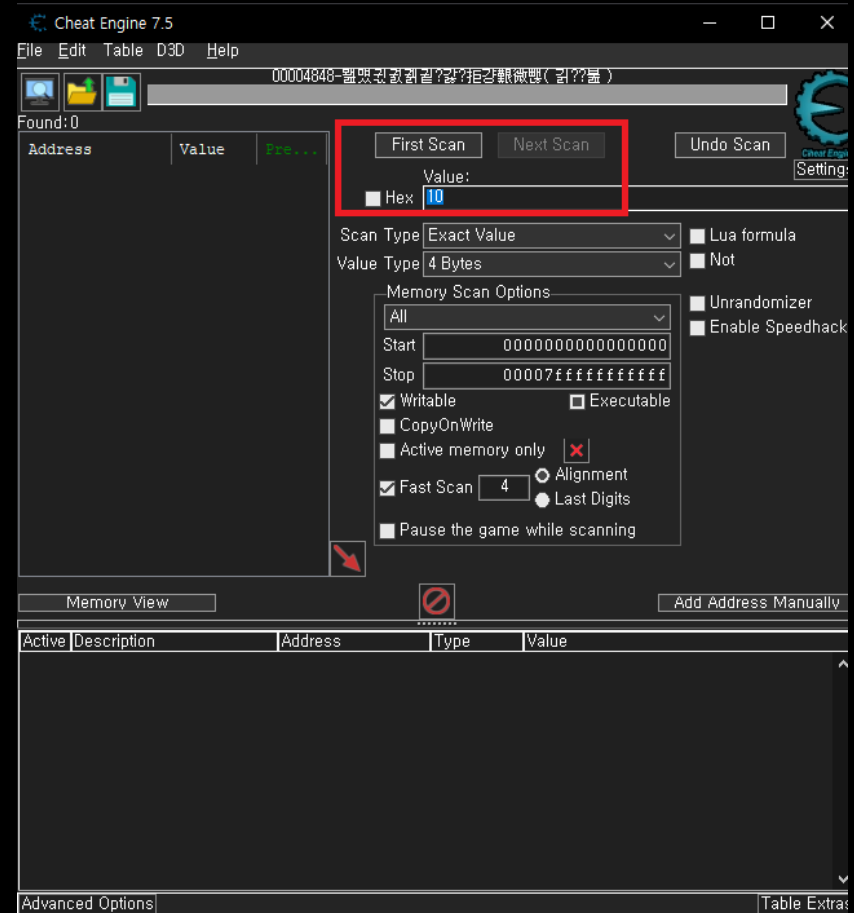
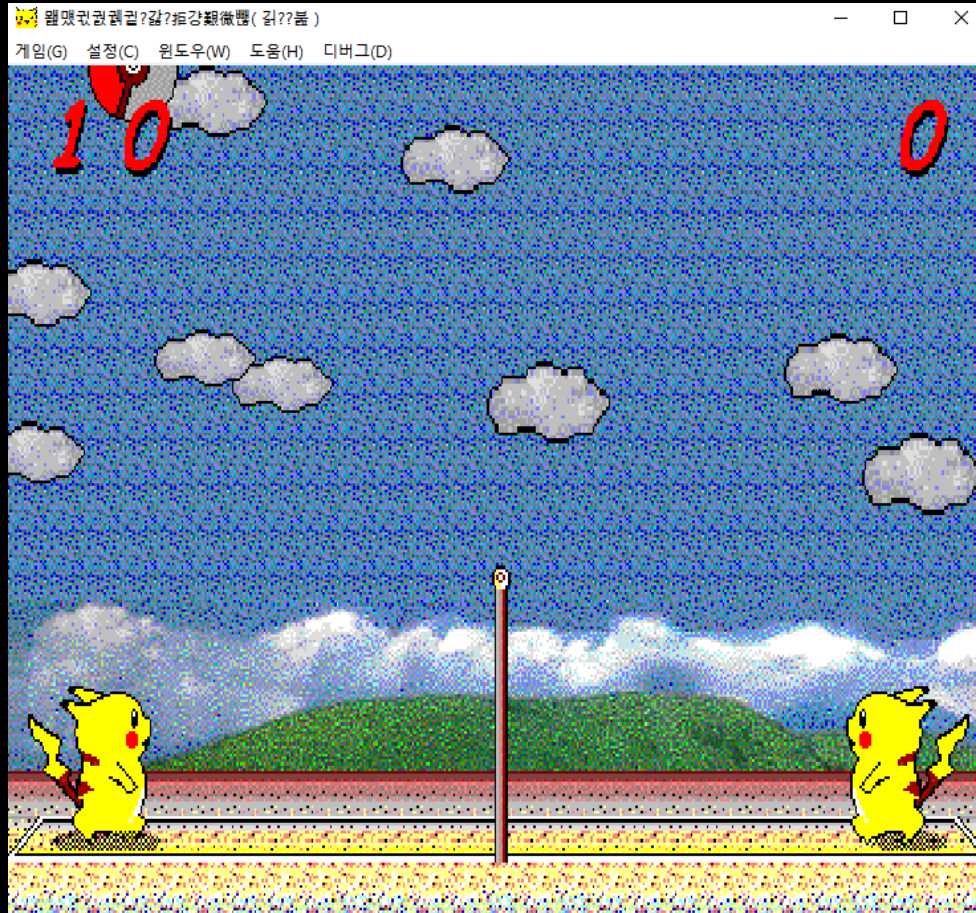


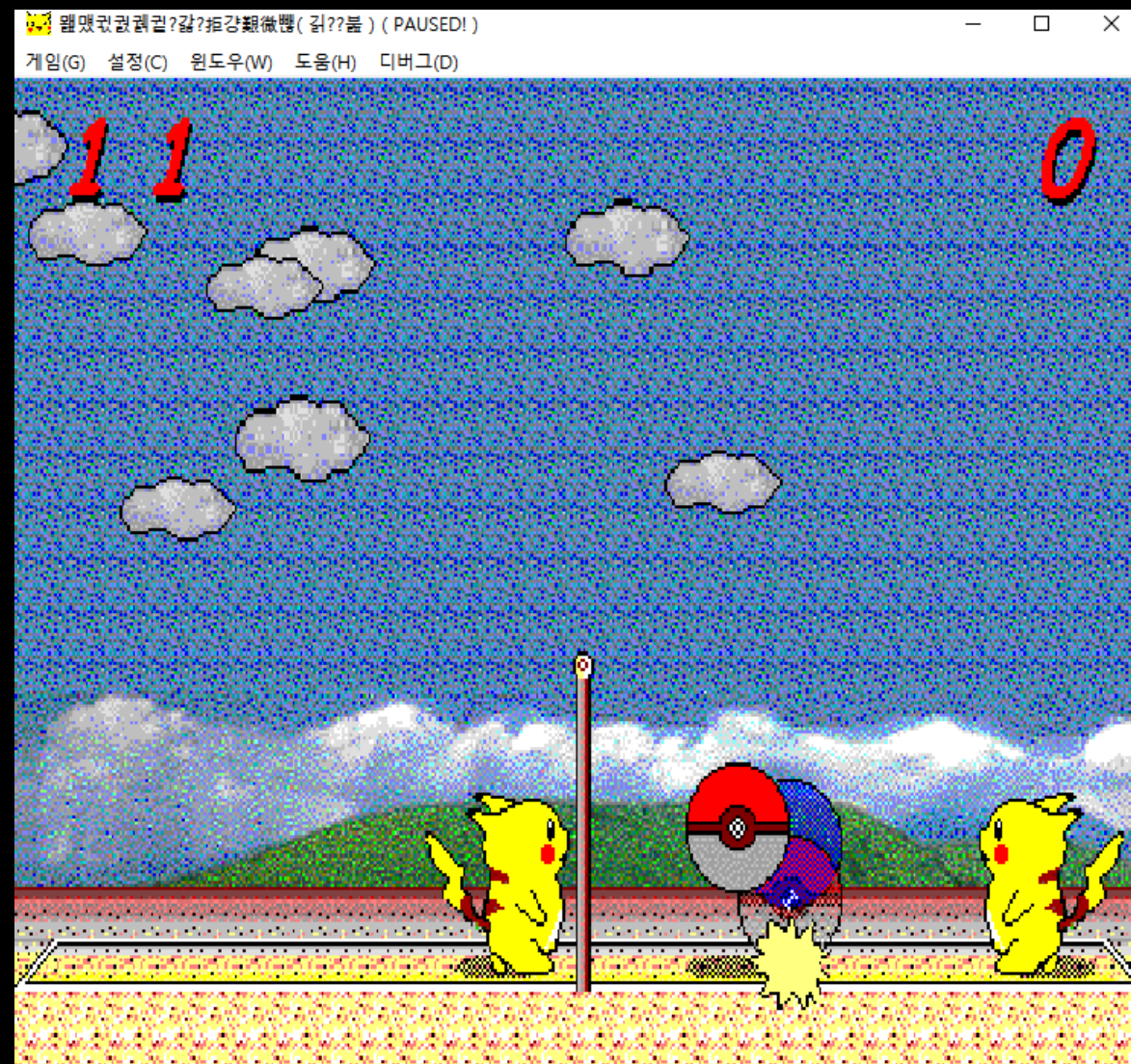
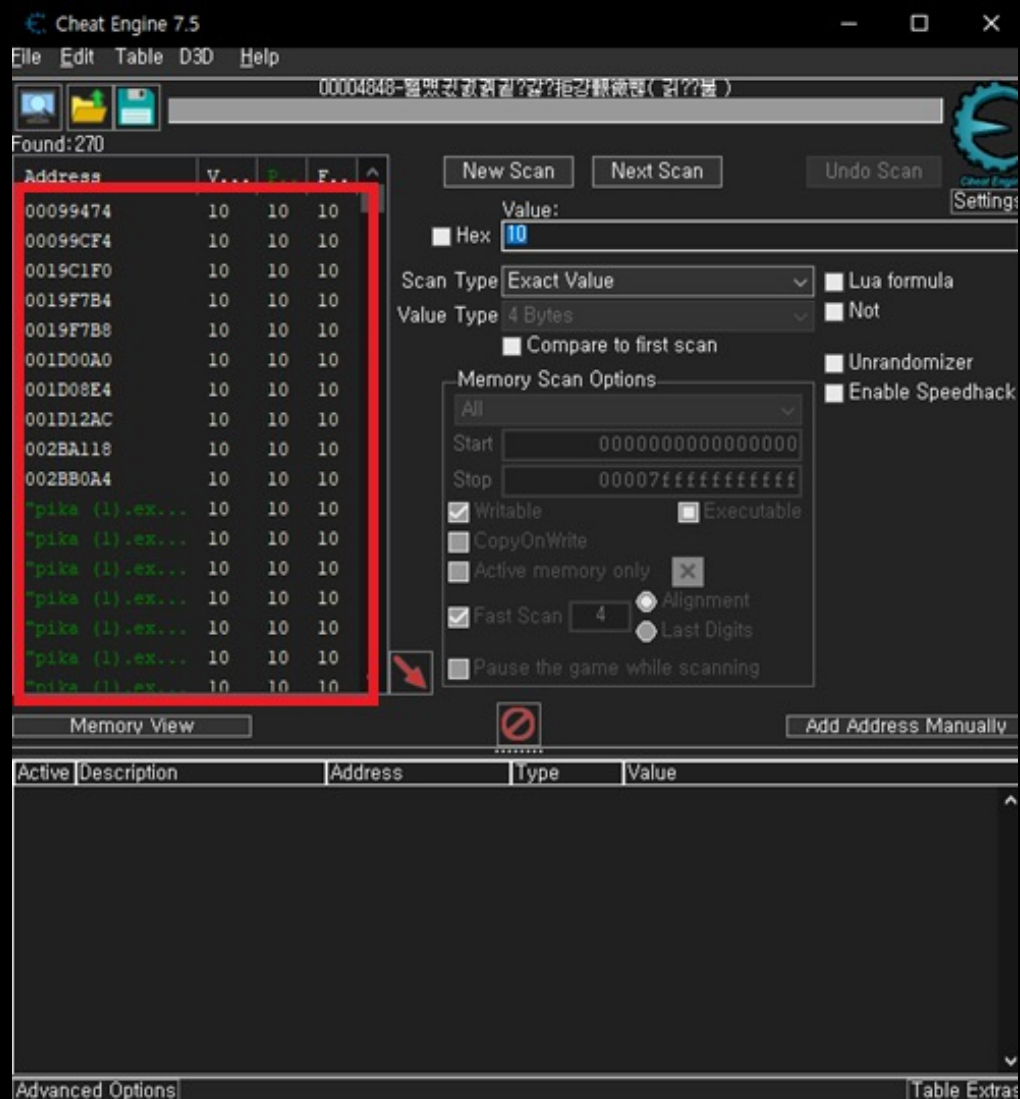
컴퓨터를 이겨보자...!

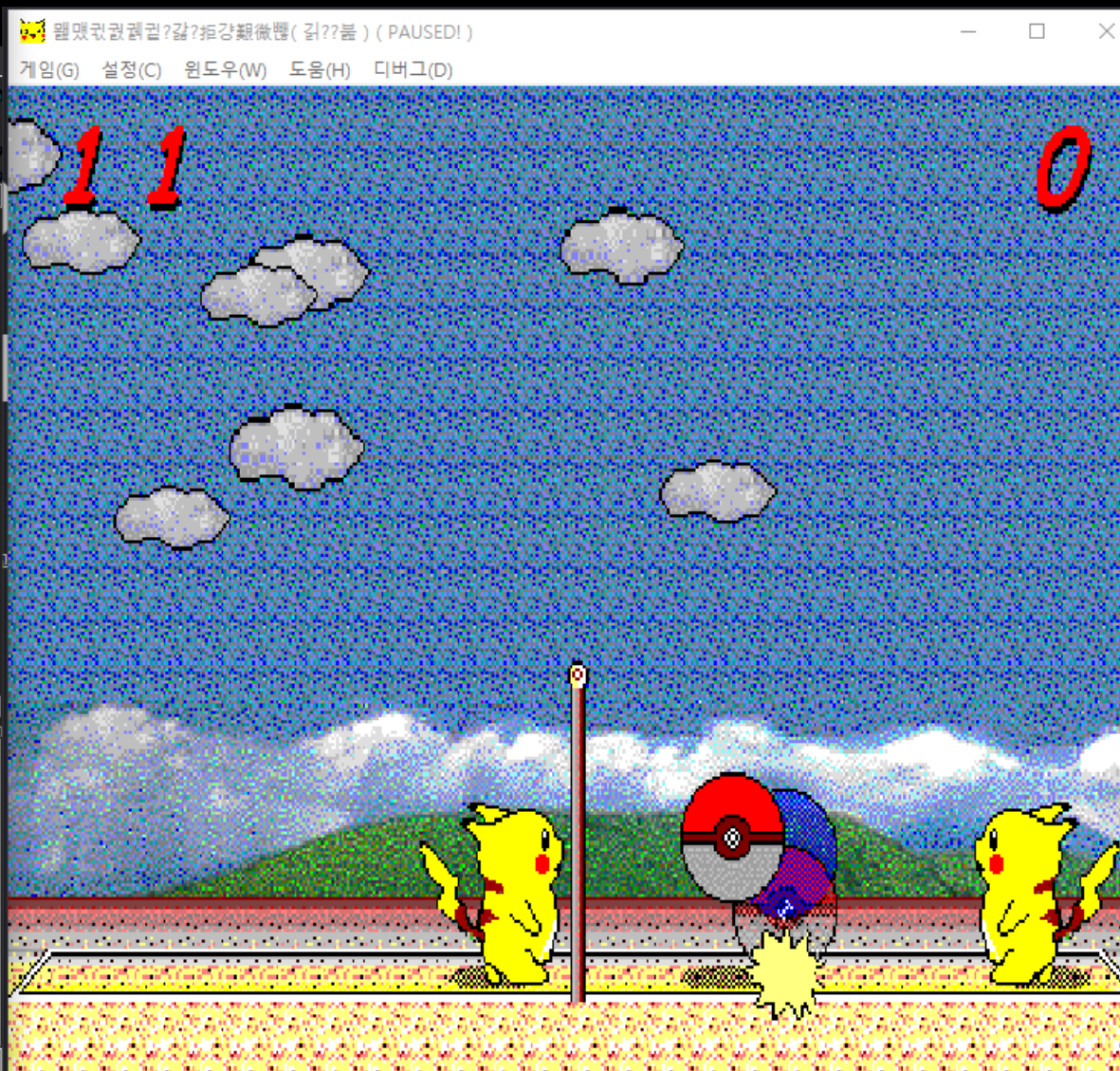
- 게임을 시작하면 컴퓨터는 신나게 서브하여 점수를 올린다.
- 여기서 가장 먼저 보이는 숫자는 10이라는 숫자다.
- 여기서 10은 컴퓨터의 점수를 의미하는걸 알 수 있다.
- 그럼 이 값을 변경하면 컴퓨터의 점수가 변경되지 않을까 하는 생각을 할 수 있다.



값을 변조해보자.

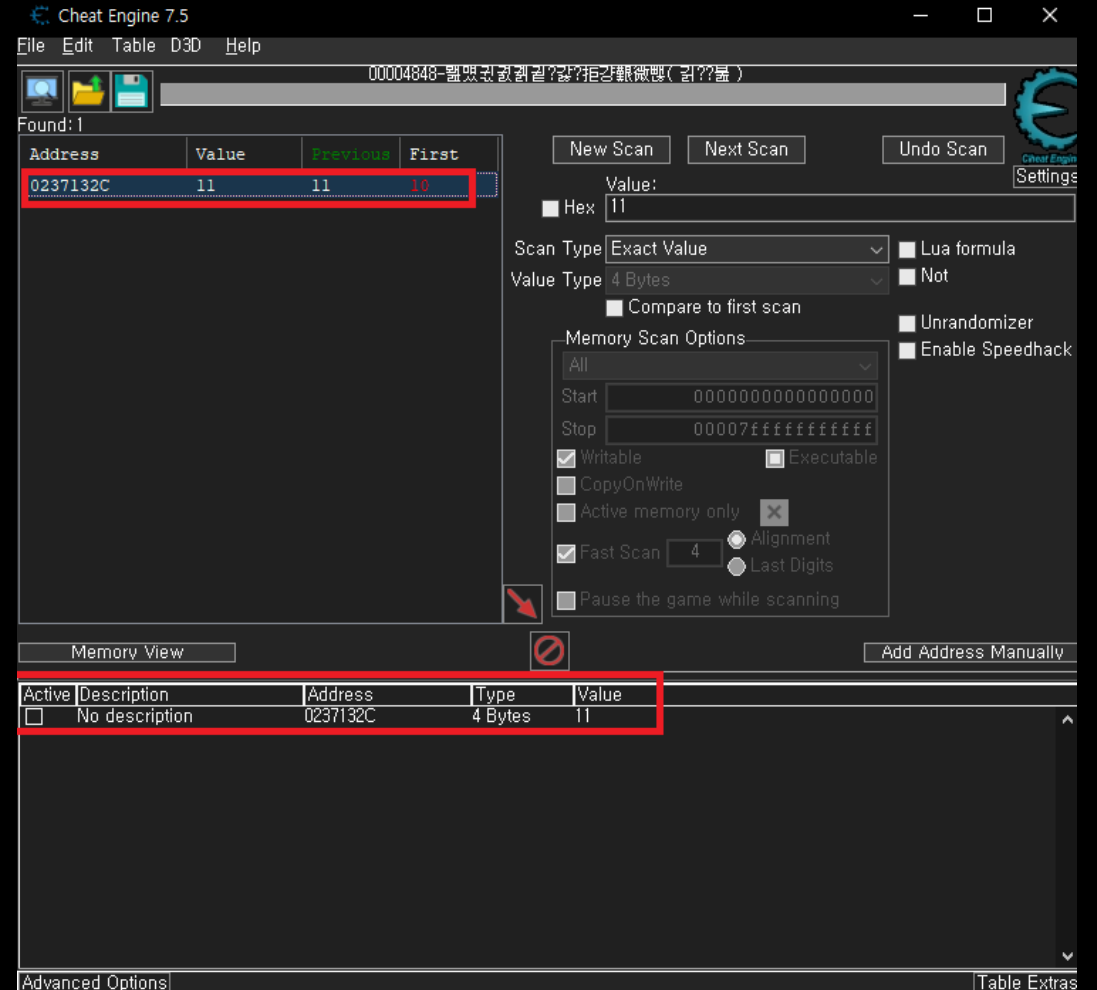






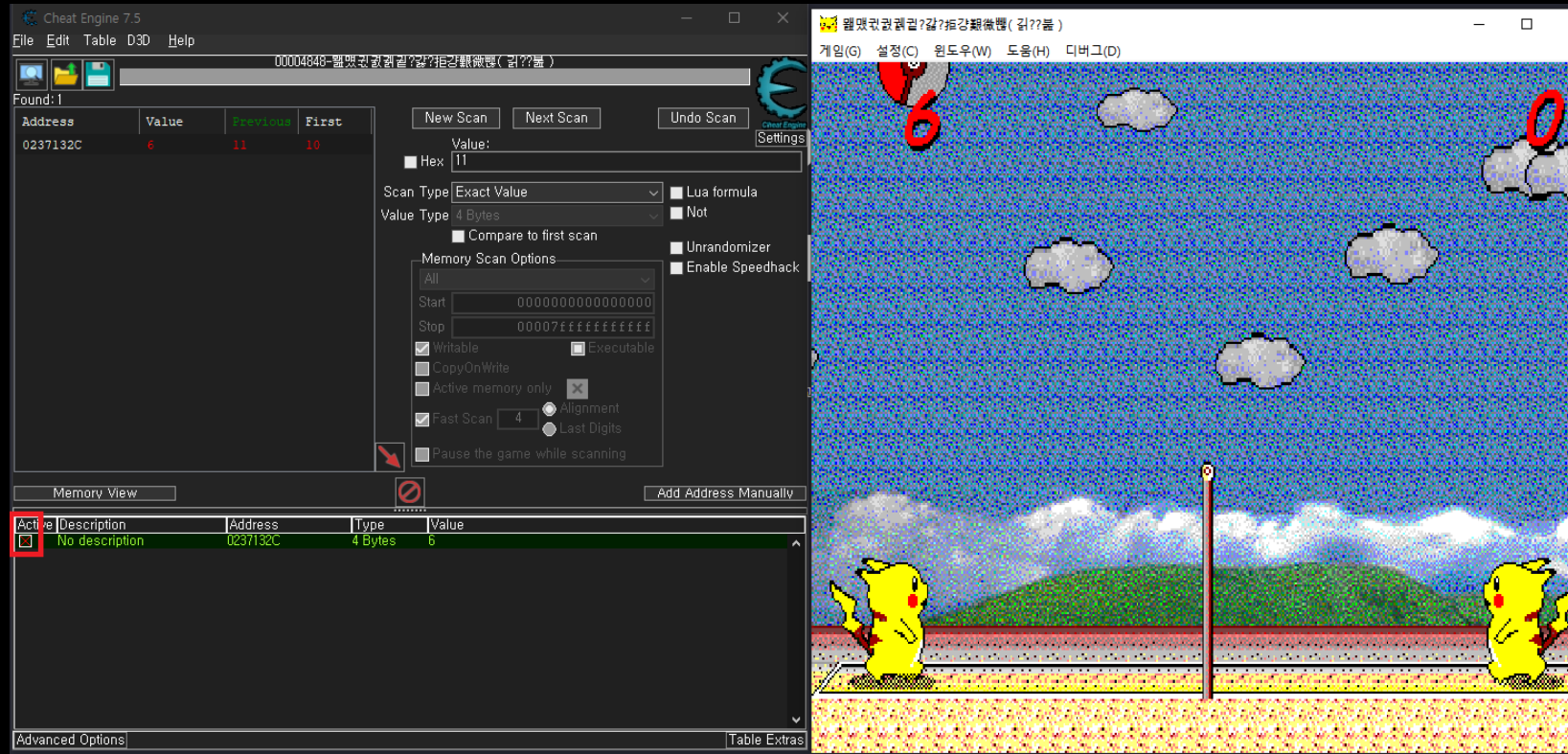
메모리 번조

- 이제 건방진 피카츄의 점수를 바꿔보자.
- 주소 부분을 더블클릭 하면 아랫 부분에 주소와 타입, 값이 나오는데, Value를 5로 수정하고 게임을 한번 봐보자.



메모리 번조

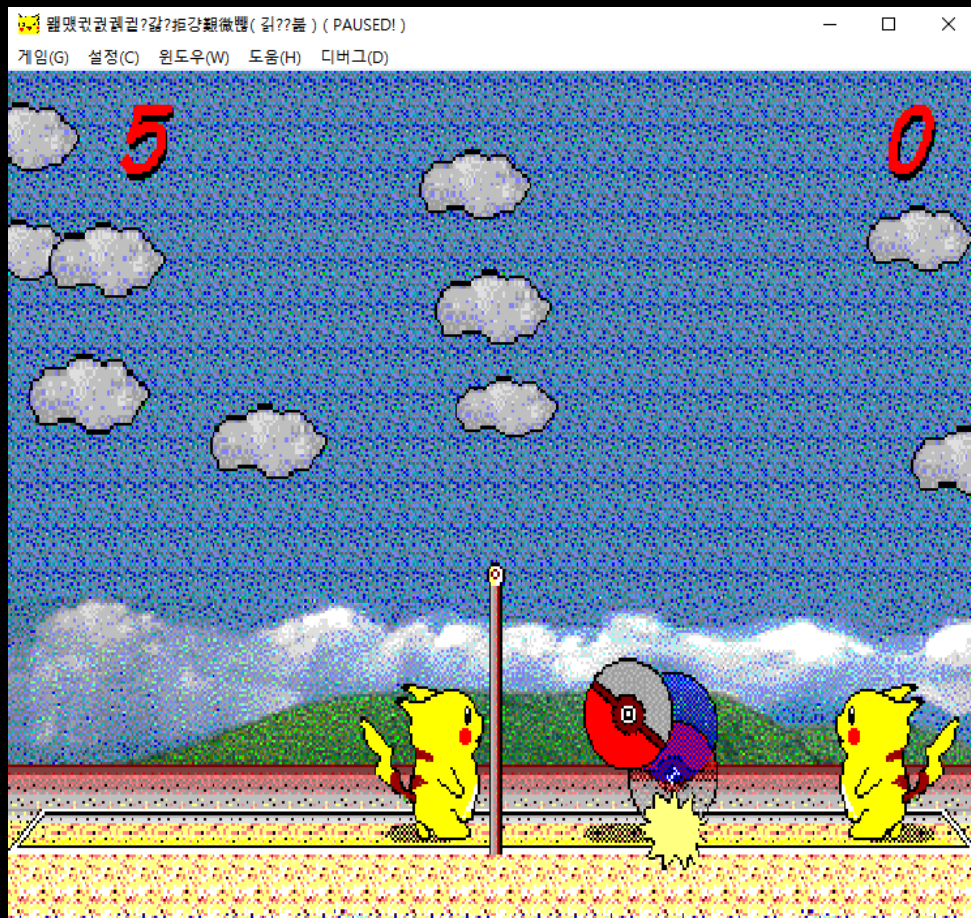
- 5로 변경하면 상대 피카츄가 5점으로 변경된 것을 확인할 수 있다.
- 그래도 점수가 계속 올라간다.
- 해당 값을 고정시키고 싶으면 치트엔진 하단의 체크박스를 표시하면 된다.



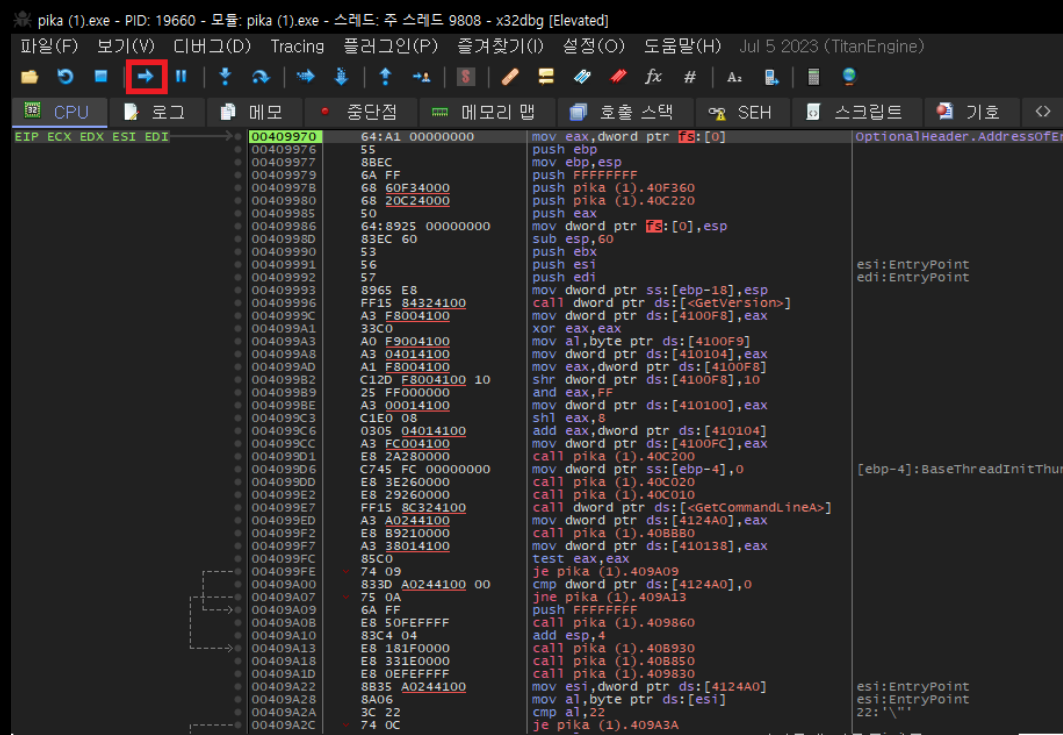
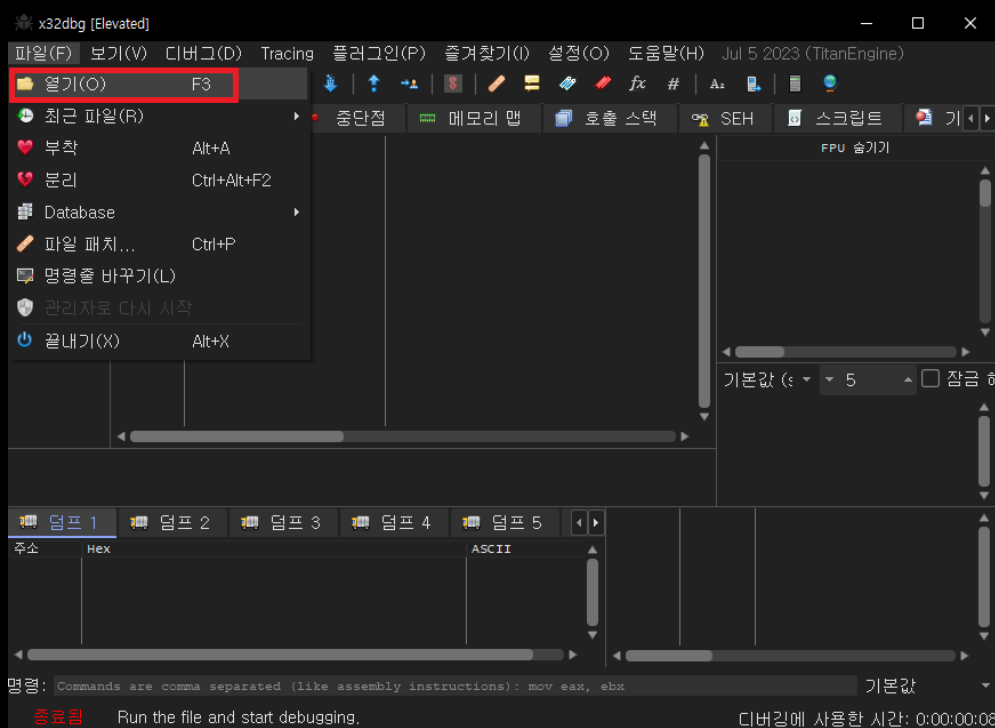
한발 더 나아가기

- 여기서는 단순히 점수만 변조했다.
- 하지만 피카츄 배구라는 게임은 점수 뿐 아니라 모든 요소가 메모리라고 볼 수 있다.
- 우리는 Cheat Engine을 가지고 메모리를 변조할 수 있음을 확인했다.
- 그렇다면 점수 뿐 아니라 다른 요소들도 변조할 수 있지 않을까...?’
- 어떤걸 변경할 수 있을까?

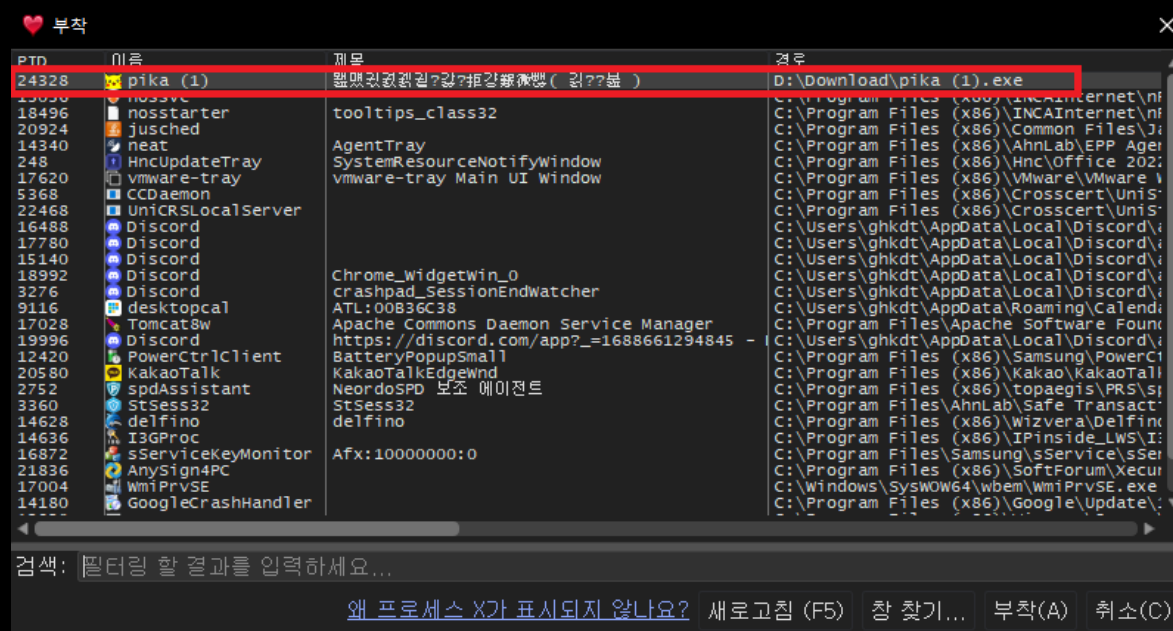
한발 더 나아가기



피카츄 배구를 동적 디버깅



- 이전 작업은 프로세스를 새로 실행시켜야 하기 때문에 주소가 서로 다르다.
- 그렇기에 프로세스를 새로 실행시키는게 아닌 프로세스에 attach한다.

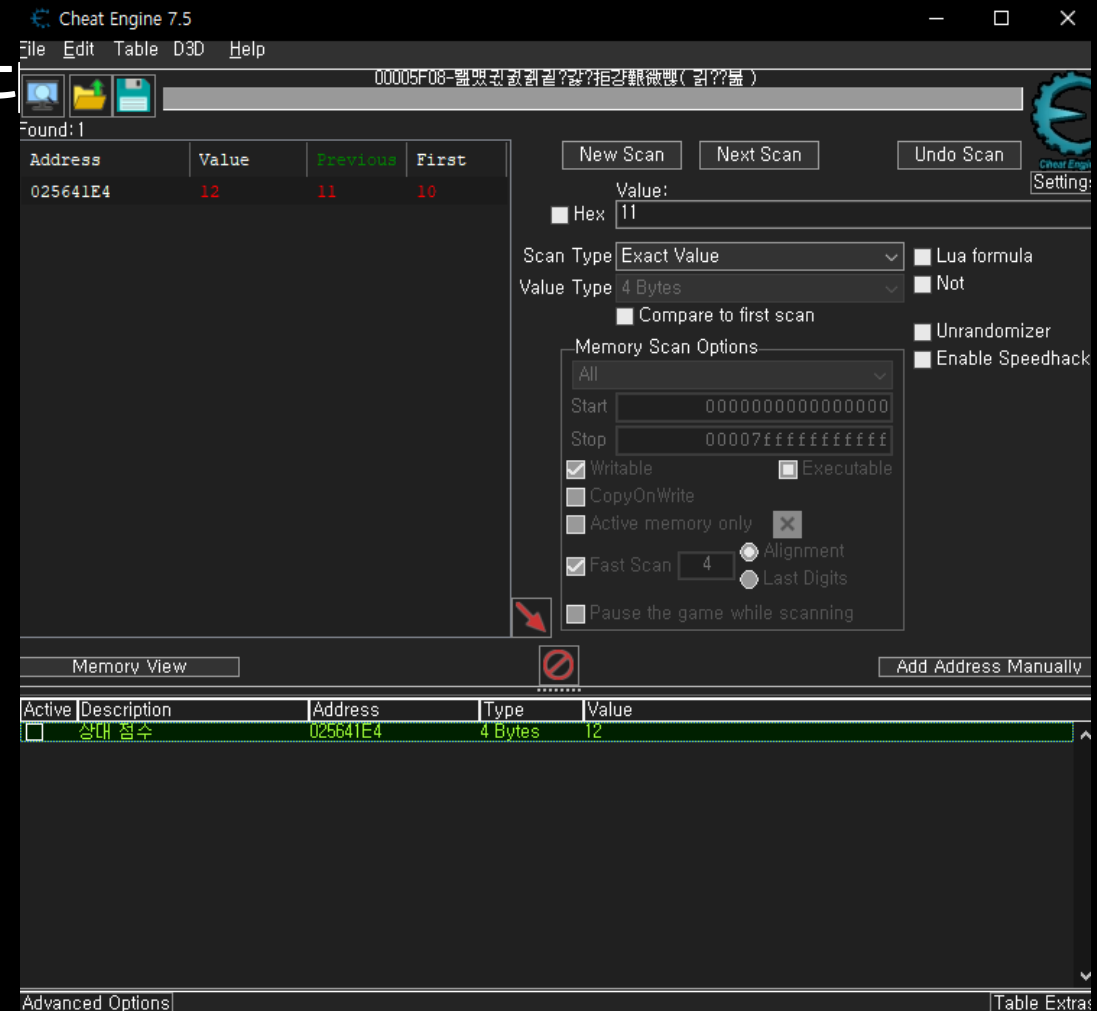


동적 디버깅 vs 정적 디버깅?

- 동적 디버깅 : 프로그램을 실행시키면서 디버깅
- 정적 디버깅 : 프로그램이 멈춰있는 상태에서 디버깅

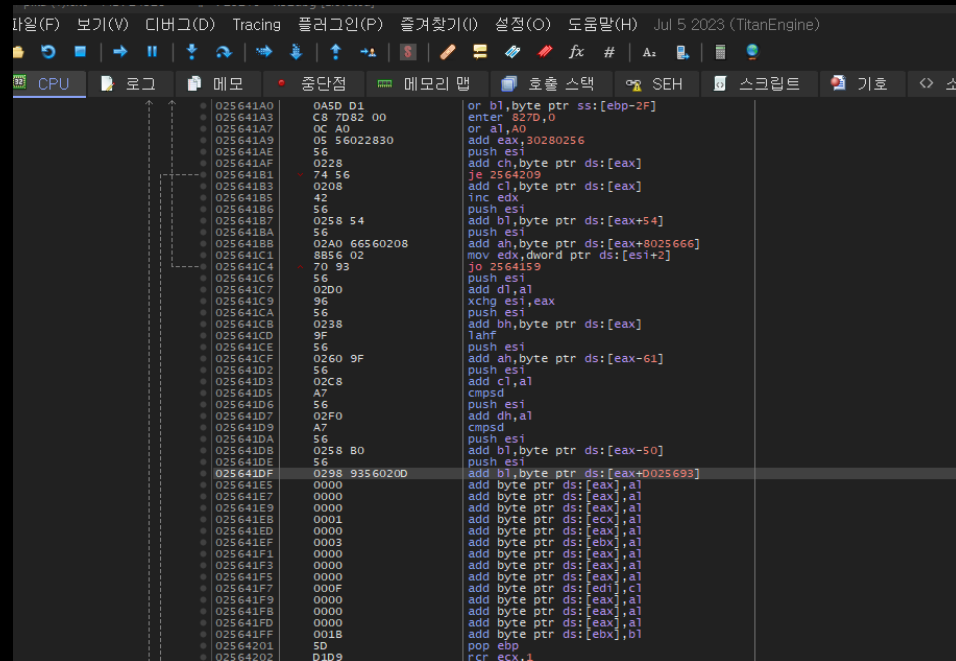
치트엔진 + 동적 디버깅

- X64dbg와 치트엔진으로 보다 섬세한 분석을 진행한다
- 다시 피카츄 배구 게임으로 돌아가서 주소를 찾는다.
- 다시 점수에 대한 주소를 찾으면 0x25641E4임을 알 수 있다.



치트엔진 + 동적 디버깅

- ctrl + g를 눌러 해당 주소로 점프한다.



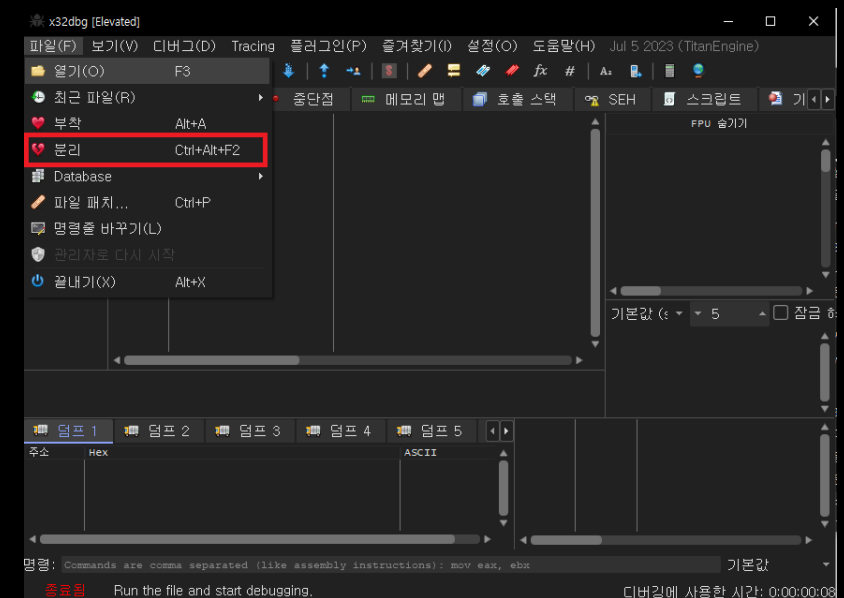
```
025641A0 0A5D D1      or bl,byte ptr ss:[ebp-2F]
025641A3 C8 7D82 00   enter 827D,0
025641A7 0C A0        or al,A0
025641A9 05 56022830  add eax,30280256
025641AE 56          push esi
025641AF 0228        add ch,byte ptr ds:[eax]
025641B1 74 56       je 2564209
025641B3 0208        add cl,byte ptr ds:[eax]
025641B5 42          inc edx
025641B6 56          push esi
025641B7 0258 54     add bl,byte ptr ds:[eax+54]
025641BA 56          push esi
025641BB 02A0 66560208 add ah,byte ptr ds:[eax+8025666]
025641C1 8856 02     mov edx,dword ptr ds:[esi+2]
025641C4 70 93       jo 2564159
025641C6 56          push esi
025641C7 02D0        add dl,al
025641C9 96          xchg esi,eax
025641CA 56          push esi
025641CB 0238        add bh,byte ptr ds:[eax]
025641CD 9F          lahf
025641CE 56          push esi
025641CF 0260 9F     add ah,byte ptr ds:[eax-61]
025641D2 56          push esi
025641D3 02C8        add cl,al
025641D6 56          cmpsd
025641D7 02F0        add dh,al
025641D9 A7          cmpsd
025641DA 56          push esi
025641DB 0258 B0     add bl,byte ptr ds:[eax-50]
025641DE 56          push esi
025641DF 0238 93560200 add bl,byte ptr ds:[eax+00256993]
025641E5 0000        add byte ptr ds:[eax],al
025641E7 0000        add byte ptr ds:[eax],al
025641E9 0000        add byte ptr ds:[eax],al
025641EB 0001        add byte ptr ds:[ecx],al
025641ED 0000        add byte ptr ds:[eax],al
025641EF 0003        add byte ptr ds:[ebx],al
025641F1 0000        add byte ptr ds:[eax],al
025641F3 0000        add byte ptr ds:[eax],al
025641F5 0000        add byte ptr ds:[eax],al
025641F7 000F        add byte ptr ds:[edi],cl
025641F9 0000        add byte ptr ds:[eax],al
025641FB 0000        add byte ptr ds:[eax],al
025641FD 0000        add byte ptr ds:[eax],al
025641FF 001B        add byte ptr ds:[ebx],bl
02564201 5D          pop ebp
02564202 D1D9        rcr ecx,1
```

따라갈 표현식 입력...

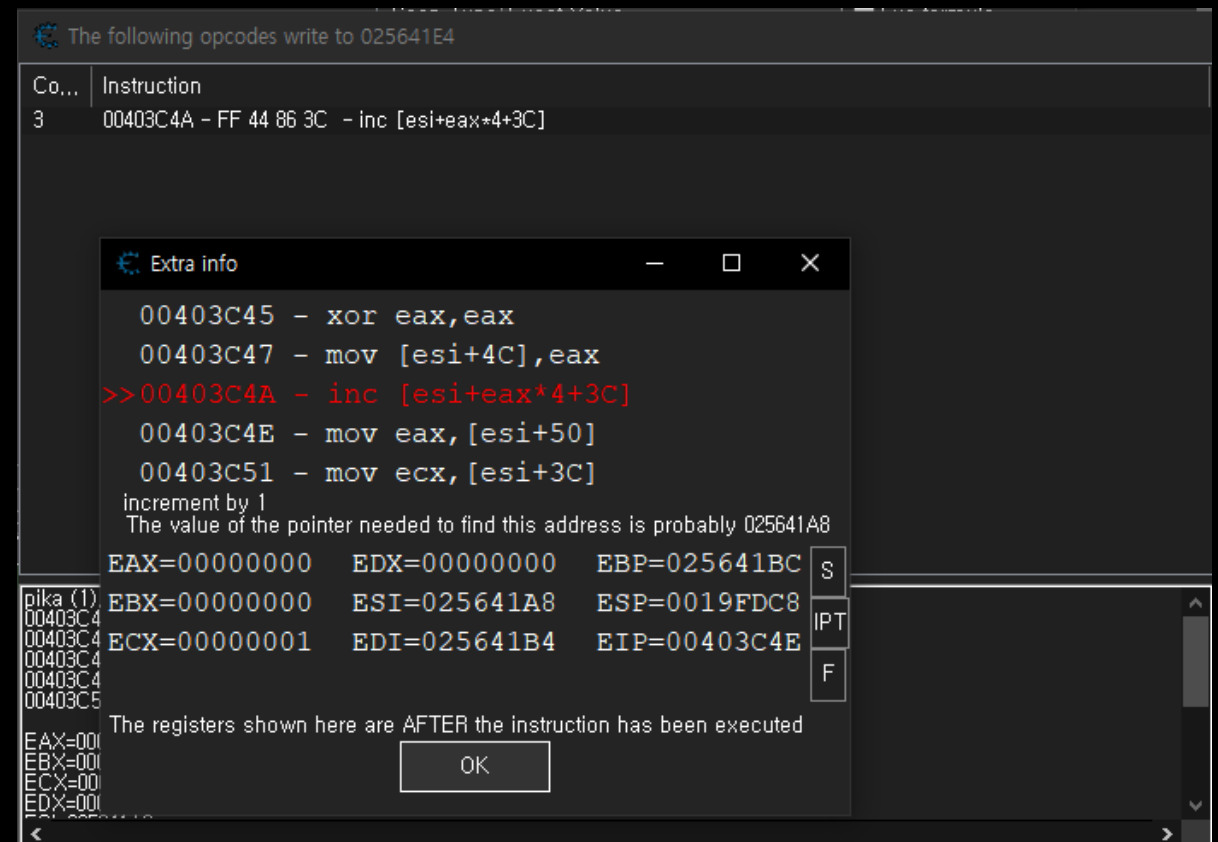
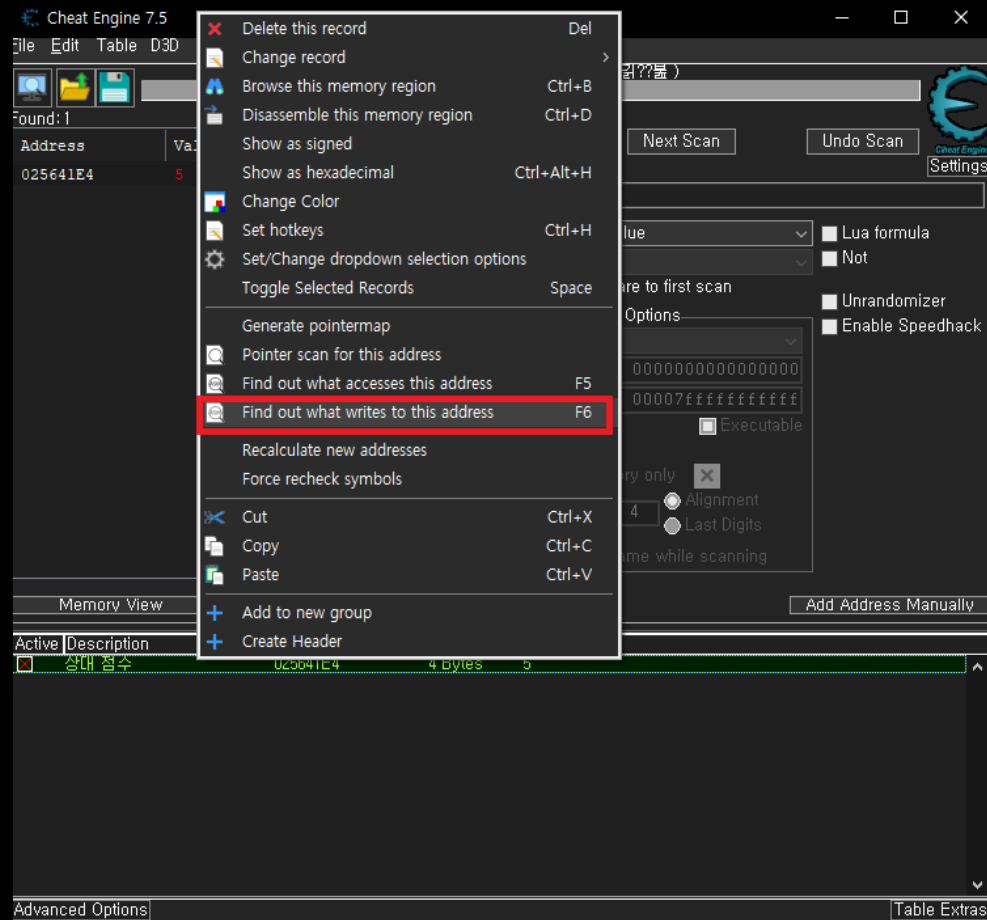
25641e4

표현식 수정! -> 025641E4

확인(O) 취소(C)



치트엔진 + 동적 디버깅



Cheat Engine 7.5

File Edit Table D3D Help

00005F08 - 알렉산더의 비밀? 2017년 12월 12일

Found: 1

Address	Value	Previous	First
025641E8	3	13	12

New Scan

Value 13

Hex

Scan Type Exact

Value Type 4 Bytes

Memory Scan

All

Start

Stop

☒ Writable

☐ Copy On Write

☐ Active Memory

☒ Fast Scan

☐ Pause the scan

Memory View

Active	Description	Address	Type	Value
☒	상대 주소	025641E4	4 Bytes	5
☐	비밀	025641E8	4 Bytes	3

Advanced Options

Table Extras

Context menu for memory address 025641E4:

- Delete this record
- Change record
- Browse this memory region
- Disassemble this memory region
- Show as signed
- Show as hexadecimal
- Change Color
- Set hotkeys
- Set/Change dropdown selection options
- Toggle Selected Records
- Generate pointermap
- Pointer scan for this address
- Find out what accesses this address**
- Find out what writes to this address
- Recalculate new addresses
- Force recheck symbols
- Cut
- Copy
- Paste
- Add to new group
- Create Header

The following opcodes accessed 025641E4

Co...	Instruction
60	00403D4E - 8B 46 3C - mov eax,[esi+3C]
1	00403C51 - 8B 4E 3C - mov ecx,[esi+3C]

Extra info

00403D46 - call 00402EC0

00403D4B - mov ecx,[esi+40]

>>00403D4E - mov eax,[esi+3C]

00403D51 - push ecx

00403D52 - push eax

copy memory

The value of the pointer needed to find this address is probably 025641A8

EAX=00000005 EDX=00000000 EBP=0000010D

EBX=0040F2C0 ESI=025641A8 ESP=0019FDC8

ECX=00000003 EDI=025641BC EIP=00403D51

The registers shown here are AFTER the instruction has been executed

OK

Stop

Cheat Engine 7.5

File Edit Table D3D Help

00005F08 - 00000000

Found: 1

Address	Value	Previous	First
025641E8	3	13	12

Scan Type: E

Value Type: 4

Memory View

Active	Description	Address	Type	Value
☒	상대 점수	025641E4	4 Bytes	5
☒	내 점수	025641E8	4 Bytes	3

Advanced Options

Table Extras

Context Menu:

- Delete this record (Del)
- Change record (>)
- Browse this memory region (Ctrl+B)
- Disassemble this memory region (Ctrl+D)
- Show as signed
- Show as hexadecimal (Ctrl+Alt+H)
- Change Color
- Set hotkeys (Ctrl+H)
- Set/Change dropdown selection options
- Toggle Selected Records (Space)
- Generate pointermap
- Pointer scan for this address
- Find out what accesses this address (F5)
- Find out what writes to this address (F6)
- Recalculate new addresses
- Force recheck symbols
- Cut (Ctrl+X)
- Copy (Ctrl+C)
- Paste (Ctrl+V)
- Add to new group
- Create Header

The following opcodes accessed 025641E8

Co...	Instruction
49	00403D4B - 8B 4E 40 - mov ecx,[esi+40]

Extra info

00403D43 - mov ecx,[esi+14]
00403D46 - call 00402EC0
>>00403D4B - mov ecx,[esi+40]
00403D4E - mov eax,[esi+3C]
00403D51 - push ecx

copy memory
The value of the pointer needed to find this address is probably 025641A8

EAX=00000001 EDX=00000000 EBP=0000010D
EBX=0040F2C0 ESI=025641A8 ESP=0019FDC8
ECX=00000003 EDI=025641BC EIP=00403D4E

The registers shown here are AFTER the instruction has been executed

OK

Stop

?

- 즉, 상대 피카츄의 점수가 +1 (증가) 하는 부분의 코드라는 뜻이다.
- 옆의 $esi + eax * 4 + 3C$ 는 뭔지 한번 알아보자.

$esi + eax * 4 + 3C$

- $if(eax == 0 \ \&\& \ esi == 1)$
 $\Rightarrow eax * 4 + 3C(4 + 3C)$
 $\Rightarrow 0x40$
- $if(eax == 1 \ \&\& \ esi == 0)$
 $\Rightarrow esi + 3C(1 + 3C)$
 - $\Rightarrow 0x3D$
- eax : return 값을 저장하거나, 산술 연산 결과 값을 저장
- esi : x32 환경에서 쓰이는 레지스

나만 점수 올라가게 설정

- 그렇다면 한번 0x3C, 0x40을 한번씩 넣어 확인해보자.

Memory Viewer

File Search View Debug Tools Kernel tools

pika (1).exe+3C2D

Address	Bytes	Opcode	Comment
pika (1).exe+3C2D	8B 4E 14	mov ecx,[esi+14]	
pika (1).exe+3C30	50	push eax	
pika (1).exe+3C31	E8 3AF1FFFF	call pika (1).exe+2D70	->pika (1).exe+2D70
pika (1).exe+3C36	81 7C 24 20 D8000000	cmp [esp+20],000000D8	216
pika (1).exe+3C3E	B8 01000000	mov eax,00000001	1
pika (1).exe+3C43	7E 02	jle pika (1).exe+3C47	->pika (1).exe+3C47
pika (1).exe+3C45	33 C0	xor eax,eax	
pika (1).exe+3C47	89 46 4C	mov [esi+4C],eax	
pika (1).exe+3C4A	FF 46 40	inc [esi+40]	
pika (1).exe+3C4D	90	nop	
pika (1).exe+3C4E	8B 46 50	mov eax,[esi+50]	
pika (1).exe+3C51	8B 4E 3C	mov ecx,[esi+3C]	
pika (1).exe+3C54	3B C8	cmp ecx,eax	

increment by 1

Protect:Read/Write AllocationBase=02560000 Base=02564000 Size=B000

address	E4	E5	E6	E7	E8	E9	EA	EB	EC	ED	EE	EF	F0	F1	F2	F3	F4	F5	F6	F7	F8	F9	FA	FB	45	67	89	AB	CD	EF	01	23	45	67	89	AB
025641E4	05	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
025641FC	00	00	00	00	1B	5D	D1	D9	72	82	00	0C	F0	42	56	02	58	4B	56	02	80	4B	56	02												
02564214	A8	4B	56	02	D0	4B	56	02	F8	4B	56	02	20	4C	56	02	48	4C	56	02	70	4C	56	02	KV.	KV.	KV.	LV.	HLV.	pLV.						
0256422C	98	4C	56	02	C0	4C	56	02	E8	4C	56	02	10	4D	56	02	38	4D	56	02	60	4D	56	02	LV.	LV.	LV.	MV.	8MV.	MV.						
02564244	88	4D	56	02	B0	4D	56	02	D8	4D	56	02	00	4E	56	02	28	4E	56	02	50	4E	56	02	MV.	MV.	MV.	NV.	(NV.	PNV.						
0256425C	78	4E	56	02	A0	4E	56	02	C8	4E	56	02	F0	4E	56	02	18	4F	56	02	40	4F	56	02	xNV.	NV.	NV.	NV.	..OV.	@OV.						
02564274	68	4F	56	02	90	4F	56	02	B8	4F	56	02	E8	4F	56	02	10	50	56	02	38	50	56	02	hOV.	OV.	OV.	OV.	..EV.	@EV.						
0256428C	60	50	56	02	88	50	56	02	B0	50	56	02	10	54	56	02	28	54	56	02	40	54	56	02	^PV.	PV.	PV.	TV.	(TV.	@TV.						
025642A4	D8	50	56	02	00	50	56	02	01	00	00	00	B8	00	00	00	F4	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
025642BC	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	01	00	00	00	01	00	00	00												
025642D4	01	00	00	00	00	00	00	00	00	00	00	00	04	00	00	00	00	00	00	00	00	00	00	00	81	5D	D1	43								
025642EC	63	82	00	0C	70	F0	40	00	00	00	00	00	28	47	56	02	68	0C	28	02	40	01	00	00	c	..p	@.		(GV.	h.	(.	@.				
02564304	D0	01	00	00	00	00	00	00	00	00	00	00	40	01	00	00	D0	01	00	00	00	00	00	00												
0256431C	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00												



Step2. 계속 배구하기

공이 바닥에 닿으면 점수가 올라간다.

하지만 바닥에 닿아도 점수가 증가하지 않도록 하고자 한다.

이에 대한 루틴은 점수가 오르기 전 과정에 있을 거라는 추측을 해본다.

그리고 또 한가지 생각을 해 볼 수 있다.

공이 바닥에 닿으면 1라운드가 증가하고 승자의 점수가 증가한다.

그렇다는건 공이 바닥에 닿을 때의 조건도 분명 존재할 것이다.

어셈블리에서 조건문을 실행할 때, jle, jne 등이 있는데,

inc 에 가까운 부분부터 일일이 breakpoint 를 걸며 확인해 보았다.

Memory Viewer - Currently debugging thread 5FDC

File Search View Debug Tools Kernel tools

Toggle Breakpoint Run Step Into Step Over Step Out Run till...

Address	Bytes	Opcode	Comment
pika (1).exe+3C2D	8B 4E 14	mov ecx,[esi+14]	
pika (1).exe+3C30	5D	push eax	
pika (1).exe+3C31	E8 3AF1FFFF	call pika (1).exe+2D70	->pika (1)
pika (1).exe+3C36	81 7C 24 20 D8000000	cmp [esp+20],000000D8	216
pika (1).exe+3C3E	B8 01000000	mov eax,00000001	1
pika (1).exe+3C43	7E 02	jle pika (1).exe+3C45	
pika (1).exe+3C45	33 C0	xor eax,eax	
pika (1).exe+3C47	89 46 4C	mov [esi+4C],eax	
pika (1).exe+3C4A	FF 46 40	inc [esi+40]	
pika (1).exe+3C4D	9D	nop	

jump short if less or equal (ZF=1 or SF=0)

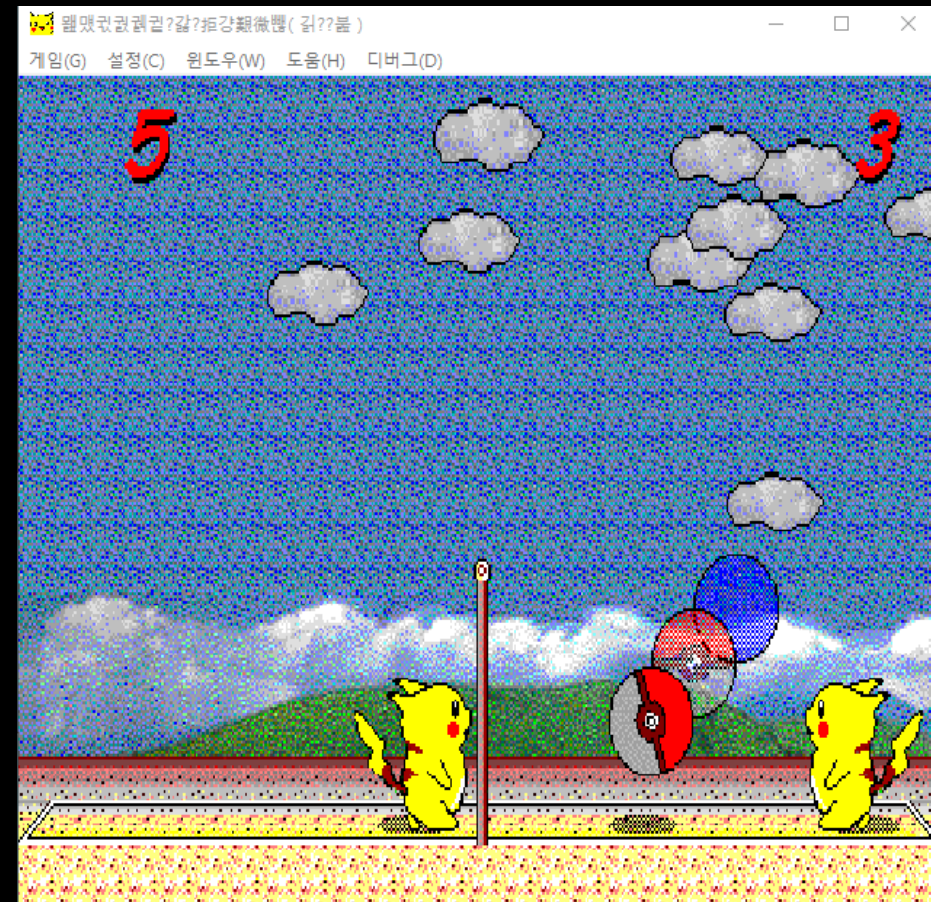
Protect:Read/Write AllocationBase=02560000 Base=02564000 Size=...

address	E4	E5	E6	E7	E8	E9	EA	EB	EC	ED	EE	EF	F0	F1	F2	F3	4567
025641E4	05	00	00	00	03	00	00	00	1F	00	00	00	02	00	00	00	...
025641F4	00	00	00	00	0F	00	00	00	00	00	00	00	1B	5D	D1	D9	...
02564204	72	82	00	0C	F0	42	56	02	58	4B	56	02	80	4B	56	02	r...
02564214	A8	4B	56	02	D0	4B	56	02	F8	4B	56	02	20	4C	56	02	KV...
02564224	48	4C	56	02	70	4C	56	02	98	4C	56	02	C0	4C	56	02	HLV...
02564234	E8	4C	56	02	10	4D	56	02	38	4D	56	02	60	4D	56	02	LV...
02564244	88	4D	56	02	B0	4D	56	02	D8	4D	56	02	00	4E	56	02	MV...
02564254	28	4E	56	02	50	4E	56	02	78	4E	56	02	A0	4E	56	02	(NV...
02564264	C8	4E	56	02	F0	4E	56	02	18	4F	56	02	40	4F	56	02	NV...
02564274	68	4F	56	02	90	4F	56	02	B8	4F	56	02	E8	4F	56	02	hOV...
02564284	10	50	56	02	38	50	56	02	60	50	56	02	88	50	56	02	PV...
02564294	B0	50	56	02	10	54	56	02	28	54	56	02	40	54	56	02	PV...
025642A4	D8	50	56	02	00	00	00	01	00	00	00	00	B8	00	00	00	PV...
025642B4	F4	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	...

Registers: EAX 00000001 CF 0 EBX 00000000 PF 1 ECX 00000001 AF 0 EDX 00000000 ZF 0 ESI 025641A8 SF 0 EDI 025641B4 DF 0 EBP 025641BC OF 0 ESP 0019FDC8

Flags

Go to address Ctrl+G Back BkSp Follow Space Set/Unset bookmark Goto bookmark Replace with code that does nothing Add to the code list Assemble Show relative addresses Ctrl+Enter Copy to clipboard Change register at this location Set breakpoint (Hardware Breakpoint) F5 Set specific breakpoint type Break and trace instructions Find out what addresses this instruction accesses Select current function Spawn diagram Shift+Ctrl+D



jle?

- Jle는 Jump Less Equal의 약자로 ~보다 작다면 점프한다는 뜻이다.
- 그렇다는 건 상대가 점수를 획득하게 되는 상황의 반대 상황으로 변경하도록 변경하면 되는데 이는 어셈블리어로 Jg(Jump Greater)이다.
- 그렇다면 Jle대신 Jg로 변경해보자.

Memory Viewer - Currently debugging thread 5FDC

File Search View Debug Tools Kernel tools

Toggle Breakpoint Run Step Into Step Over Step Out Run till...

Address Bytes Opcode Comment

pika (1).exe+3C3E B8 01000000 mov eax,00000001

>>pika (1).exe+3C43 7E 02 jle pika (1).exe+3C47 -> pika (1)

pika (1).exe+3C45 33 C0 xor eax, eax

pika (1).exe+3C47 89 46 4C mov [esi+4C], eax

pika (1).exe+3C4A FF 46 40 inc [esi+40]

pika (1).exe+3C4D 90 nop

pika (1).exe+3C4E 8B 46 50 mov eax, [esi+50]

pika (1).exe+3C51 8B 4E 3C mov ecx, [esi+3C]

pika (1).exe+3C54 3B C8 cmp ecx, eax

pika (1).exe+3C56 7D 15 inl pika (1).exe+3C5D -> pika (1)

jump short if less or equal (ZF=1 or SF=OF)

Registers: EAX 00000001 CF 0 EBX 00000000 PF 1 ECX 00000001 AF 0 EDX 00000000 ZF 0 ESI 025641A8 SF 0 EDI 025641B4 DF 0 EBP 025641BC OF 0

Special CS 0023 SS 002B DS 002B ES 002B FS 0053

Protect:Read/Write AllocationBase=02560000 Base=02564000 Size=B000

address E4 E5 E6 E7 E8 E9 EA EB EC ED EE EF F0 F1 F2 F3 456789ABCDEF0123

025641E4 05 00 00 00 03 00 00 00 1F 00 00 00 02 00 00 00]

025641F4 00 00 00 00 0F 00 00 00 00 00 00 00 1B 5D D1 D9]

02564204 72 82 00 0C F0 42 56 02 58 4B 56 02 80 4B 56 02 r..BV.XKV. KV.

02564214 A8 4B 56 02 D0 4B 56 02 F8 4B 56 02 20 4C 56 02 KV. KV. KV. LV.

02564224 48 4C 56 02 70 4C 56 02 98 4C 56 02 C0 4C 56 02 HLV. pLV. LV. LV.

02564234 E8 4C 56 02 10 4D 56 02 38 4D 56 02 60 4D 56 02 LV..MV.8MV. MV.

02564244 88 4D 56 02 B0 4D 56 02 D8 4D 56 02 00 4E 56 02 MV. MV. MV. NV.

02564254 28 4E 56 02 50 4E 56 02 78 4E 56 02 A0 4E 56 02 (NV. PNV. xNV. NV.

02564264 C8 4E 56 02 F0 4E 56 02 18 4F 56 02 40 4F 56 02 NV. NV. OV. @OV.

02564274 68 4F 56 02 90 4F 56 02 B8 4F 56 02 E8 4F 56 02 hOV. OV. OV. OV.

02564284 10 50 56 02 38 50 56 02 60 50 56 02 88 50 56 02 FV.8FV. FV. FV.

02564294 B0 50 56 02 10 54 56 02 28 54 56 02 40 54 56 02 FV..TV. (TV. @TV.

025642A4 D8 50 56 02 00 00 00 00 01 00 00 00 B8 00 00 00 FV.....

025642B4 F4 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00]

Return Address Parameters

pika (1).exe+4F60 0019FE8C.0

pika (1).exe+4E65 0019FE8C.p

pika (1).exe+9AAD pika (1).exe

pika (1).exe+6720 pika (1).exe

pika (1).exe+68B0 pika (1).exe

pika (1).exe+68C0 pika (1).exe

pika (1).exe+68D0 pika (1).exe

pika (1).exe+68E0 pika (1).exe

pika (1).exe+68F0 pika (1).exe

pika (1).exe+6900 pika (1).exe

pika (1).exe+6910 pika (1).exe

pika (1).exe+6920 pika (1).exe

pika (1).exe+6930 pika (1).exe

pika (1).exe+6940 pika (1).exe

pika (1).exe+6950 pika (1).exe

pika (1).exe+6960 pika (1).exe

pika (1).exe+6970 pika (1).exe

pika (1).exe+6980 pika (1).exe

pika (1).exe+6990 pika (1).exe

pika (1).exe+69A0 pika (1).exe

pika (1).exe+69B0 pika (1).exe

pika (1).exe+69C0 pika (1).exe

pika (1).exe+69D0 pika (1).exe

pika (1).exe+69E0 pika (1).exe

pika (1).exe+69F0 pika (1).exe

Memory Viewer - Currently debugging thread 5FDC

File Search View Debug Tools Kernel tools

Toggle Breakpoint Run Step Into Step Over Step Out Run till...

Address Bytes Opcode Comment

pika (1).exe+3C3E B8 01000000 mov eax,00000001

>>pika (1).exe+3C43 7F 02 jg pika (1).exe+3C47 ---> -> pika (1)

pika (1).exe+3C45 33 C0 xor eax, eax

pika (1).exe+3C47 89 46 4C mov [esi+4C], eax

pika (1).exe+3C4A FF 46 40 inc [esi+40]

pika (1).exe+3C4D 90 nop

pika (1).exe+3C4E 8B 46 50 mov eax, [esi+50]

pika (1).exe+3C51 8B 4E 3C mov ecx, [esi+3C]

pika (1).exe+3C54 3B C8 cmp ecx, eax

pika (1).exe+3C56 7D 15 inl pika (1).exe+3C5D -> pika (1)

jump short if greater (ZF=0 or SF=OF)

Registers: EAX 00000001 CF 0 EBX 00000000 PF 1 ECX 00000001 AF 0 EDX 00000000 ZF 0 ESI 025641A8 SF 0 EDI 025641B4 DF 0 EBP 025641BC OF 0

Special CS 0023 SS 002B DS 002B ES 002B FS 0053

Protect:Read/Write AllocationBase=02560000 Base=02564000 Size=B000

address E4 E5 E6 E7 E8 E9 EA EB EC ED EE EF F0 F1 F2 F3 456789ABCDEF0123

025641E4 05 00 00 00 03 00 00 00 1F 00 00 00 02 00 00 00]

025641F4 00 00 00 00 0F 00 00 00 00 00 00 00 1B 5D D1 D9]

02564204 72 82 00 0C F0 42 56 02 58 4B 56 02 80 4B 56 02 r..BV.XKV. KV.

02564214 A8 4B 56 02 D0 4B 56 02 F8 4B 56 02 20 4C 56 02 KV. KV. KV. LV.

02564224 48 4C 56 02 70 4C 56 02 98 4C 56 02 C0 4C 56 02 HLV. pLV. LV. LV.

02564234 E8 4C 56 02 10 4D 56 02 38 4D 56 02 60 4D 56 02 LV..MV.8MV. MV.

02564244 88 4D 56 02 B0 4D 56 02 D8 4D 56 02 00 4E 56 02 MV. MV. MV. NV.

02564254 28 4E 56 02 50 4E 56 02 78 4E 56 02 A0 4E 56 02 (NV. PNV. xNV. NV.

02564264 C8 4E 56 02 F0 4E 56 02 18 4F 56 02 40 4F 56 02 NV. NV. OV. @OV.

02564274 68 4F 56 02 90 4F 56 02 B8 4F 56 02 E8 4F 56 02 hOV. OV. OV. OV.

02564284 10 50 56 02 38 50 56 02 60 50 56 02 88 50 56 02 FV.8FV. FV. FV.

02564294 B0 50 56 02 10 54 56 02 28 54 56 02 40 54 56 02 FV..TV. (TV. @TV.

025642A4 D8 50 56 02 00 00 00 00 01 00 00 00 B8 00 00 00 FV.....

025642B4 F4 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00]

Return Address Parameters

pika (1).exe+4F60 0019FE8C.0

pika (1).exe+4E65 0019FE8C.p

pika (1).exe+9AAD pika (1).exe

pika (1).exe+6720 pika (1).exe

pika (1).exe+68B0 pika (1).exe

pika (1).exe+68C0 pika (1).exe

pika (1).exe+68D0 pika (1).exe

pika (1).exe+68E0 pika (1).exe

pika (1).exe+68F0 pika (1).exe

pika (1).exe+6900 pika (1).exe

pika (1).exe+6910 pika (1).exe

pika (1).exe+6920 pika (1).exe

pika (1).exe+6930 pika (1).exe

pika (1).exe+6940 pika (1).exe

pika (1).exe+6950 pika (1).exe

pika (1).exe+6960 pika (1).exe

pika (1).exe+6970 pika (1).exe

pika (1).exe+6980 pika (1).exe

pika (1).exe+6990 pika (1).exe

pika (1).exe+69A0 pika (1).exe

pika (1).exe+69B0 pika (1).exe

pika (1).exe+69C0 pika (1).exe

pika (1).exe+69D0 pika (1).exe

pika (1).exe+69E0 pika (1).exe

pika (1).exe+69F0 pika (1).exe

- 다음은 어디로 점프하냐인데, 공이 땅에 닿아도 계속 플레이 할 거기 때문에 그대로 진행하는 부분을 지정한다.
- BP를 걸며 한번 찾아보면 0X3C9F 위치에 있는 부분부터 멈추는 것을 확인할 수 있다.
- 그렇다면 점프 위치를 0X3C9F로 변경해보자.

Memory Viewer - Running

FileSearchViewDebugToolsKernel tools

Toggle Breakpoint

Run

Step Into

Step Over

Step Out

Run till...

pika (1).exe+3C31

Address	Bytes	Opcode	Comment
pika (1).exe+3C31	E8 3AF1FFFF	call	pika (1).exe+2D70 ->pika (1).exe+2D70
pika (1).exe+3C36	81 7C 24 20 D8000000	cmp	[esp+20],000000D8 216
pika (1).exe+3C3E	B8 01000000	mov	eax,00000001 1
>>pika (1).exe+3C43	7F 5A	jg	pika (1).exe+3C3F ---> ->pika (1).exe+3C9F
pika (1).exe+3C45	33 C0	xor	eax, eax
pika (1).exe+3C47	89 46 4C	mov	[esi+4C], eax
pika (1).exe+3C4A	FF 46 40	inc	[esi+40]
pika (1).exe+3C4D	90	nop	
pika (1).exe+3C4E	8B 46 50	mov	eax, [esi+50]
pika (1).exe+3C51	8B 4E 3C	mov	ecx, [esi+3C]
pika (1).exe+3C54	3B C8	cmp	ecx, eax
jump short if greater (ZF=0 or SF=OF)			

Protect:Read/Write AllocationBase=02560000 Base=02564000 Size=B000

address	E4	E5	E6	E7	E8	E9	EA	EB	EC	ED	EE	EF	F0	F1	F2	F3	F4	F5	F6	F7	F8	F9	FA	FB	456789ABCDEF0123456789AB
025641E4	05	00	00	00	03	00	00	00	1F	00	00	00	02	00	00	00	00	00	00	00	0F	00	00	00	
025641FC	00	00	00	00	1B	5D	D1	D9	72	82	00	0C	F0	42	56	02	58	4B	56	02	80	4B	56	02	] r .. BV.XKV. KV.
02564214	A8	4B	56	02	D0	4B	56	02	F8	4B	56	02	20	4C	56	02	48	4C	56	02	70	4C	56	02	KV. KV. KV. LV.HLV. pLV.
0256422C	98	4C	56	02	C0	4C	56	02	E8	4C	56	02	10	4D	56	02	38	4D	56	02	60	4D	56	02	LV. LV. LV. MV.8MV. MV.
02564244	88	4D	56	02	B0	4D	56	02	D8	4D	56	02	00	4E	56	02	28	4E	56	02	50	4E	56	02	MV. MV. MV. NV. (NV. PNV.
0256425C	78	4E	56	02	A0	4E	56	02	C8	4E	56	02	F0	4E	56	02	18	4F	56	02	40	4F	56	02	xNV. NV. NV. NV. OV. OV.
02564274	68	4F	56	02	90	4F	56	02	B8	4F	56	02	E8	4F	56	02	10	50	56	02	38	50	56	02	hOV. OV. OV. OV. PV.8PV.
0256428C	60	50	56	02	88	50	56	02	B0	50	56	02	10	54	56	02	28	54	56	02	40	54	56	02	hPV. PV. PV. TV. (TV. TV.
025642A4	D8	50	56	02	00	50	00	00	01	00	00	00	B8	00	00	00	F4	00	00	00	00	00	00	00	PV.
025642BC	01	00	00	00	FE	FF	FF	FF	00	00	00	00	00	00	00	00	02	00	00	00	01	00	00	00	
025642D4	02	00	00	00	00	00	00	00	00	00	00	00	01	00	00	00	00	00	00	00	81	5D	D1	43	
025642EC	63	82	00	0C	70	F0	40	00	00	00	00	00	28	47	56	02	68	0C	28	02	40	01	00	00	c ..p @..... (GV.h. (. @...
02564304	D0	01	00	00	00	00	00	00	00	00	00	00	40	01	00	00	D0	01	00	00	00	00	00	00	
0256431C	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
02564334	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0256434C	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
02564364	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0256437C	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	

Registers:Flags

EAX	00000001	CF	0
EBX	00000000	PF	1
ECX	00000001	AF	0
EDX	00000000	ZF	0
ESI	025641A8	SF	0
EDI	025641B4	DF	0
EBP	025641BC	OF	0
ESP	0019FDC8		
EIP	00403C43		

Special

CS	0023
SS	002B
DS	002B
ES	002B
FS	0053

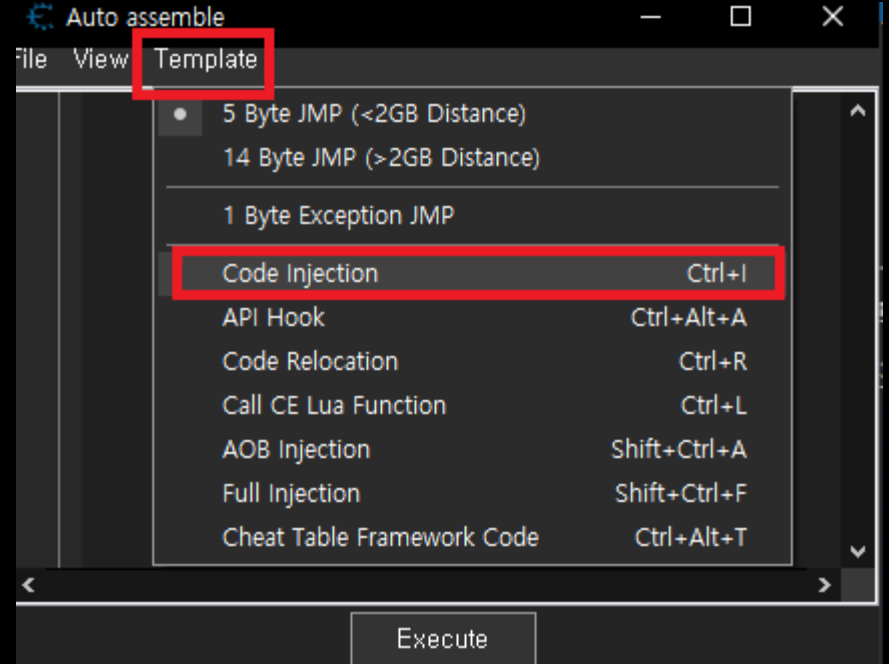
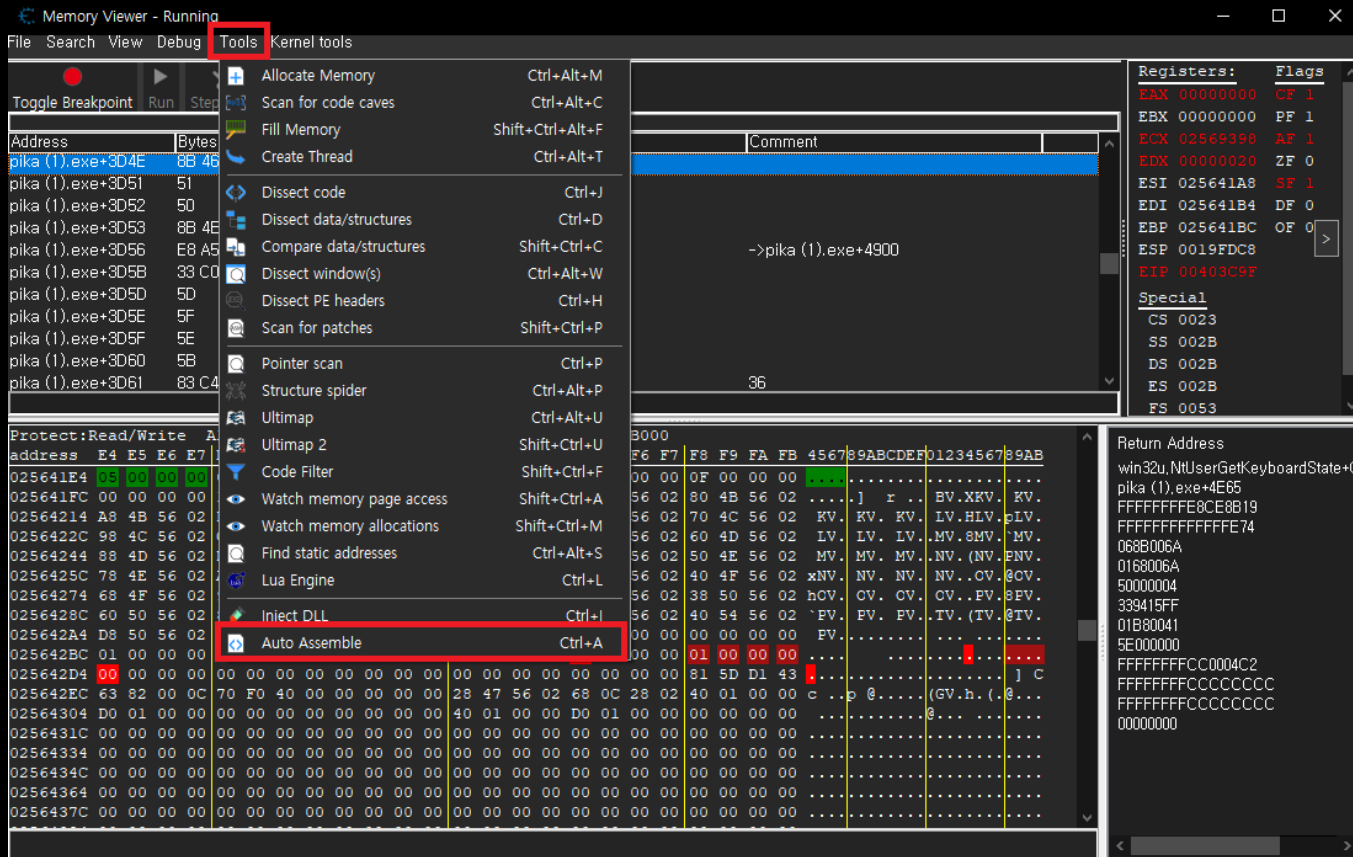
Return Address Param...

pika (1).exe+4F60	0019FE...
pika (1).exe+4E65	0019FE...
pika (1).exe+9AAD	pika (1)...
pika (1).exe+6720	pika (1)...
pika (1).exe+68B0	pika (1)...
pika (1).exe+68C0	pika (1)...
pika (1).exe+1410	pika (1)...
pika (1).exe+68F0	pika (1)...
pika (1).exe+6900	pika (1)...
pika (1).exe+6A10	pika (1)...
pika (1).exe+69A0	pika (1)...
pika (1).exe+69B0	pika (1)...
pika (1).exe+1420	pika (1)...
pika (1).exe+69D0	pika (1)...
pika (1).exe+69E0	pika (1)...
pika (1).exe+1430	pika (1)...
pika (1).exe+6A00	pika (1)...
pika (1).exe+16A0	pika (1)...
pika (1).exe+1440	pika (1)...
pika (1).exe+6A50	pika (1)...

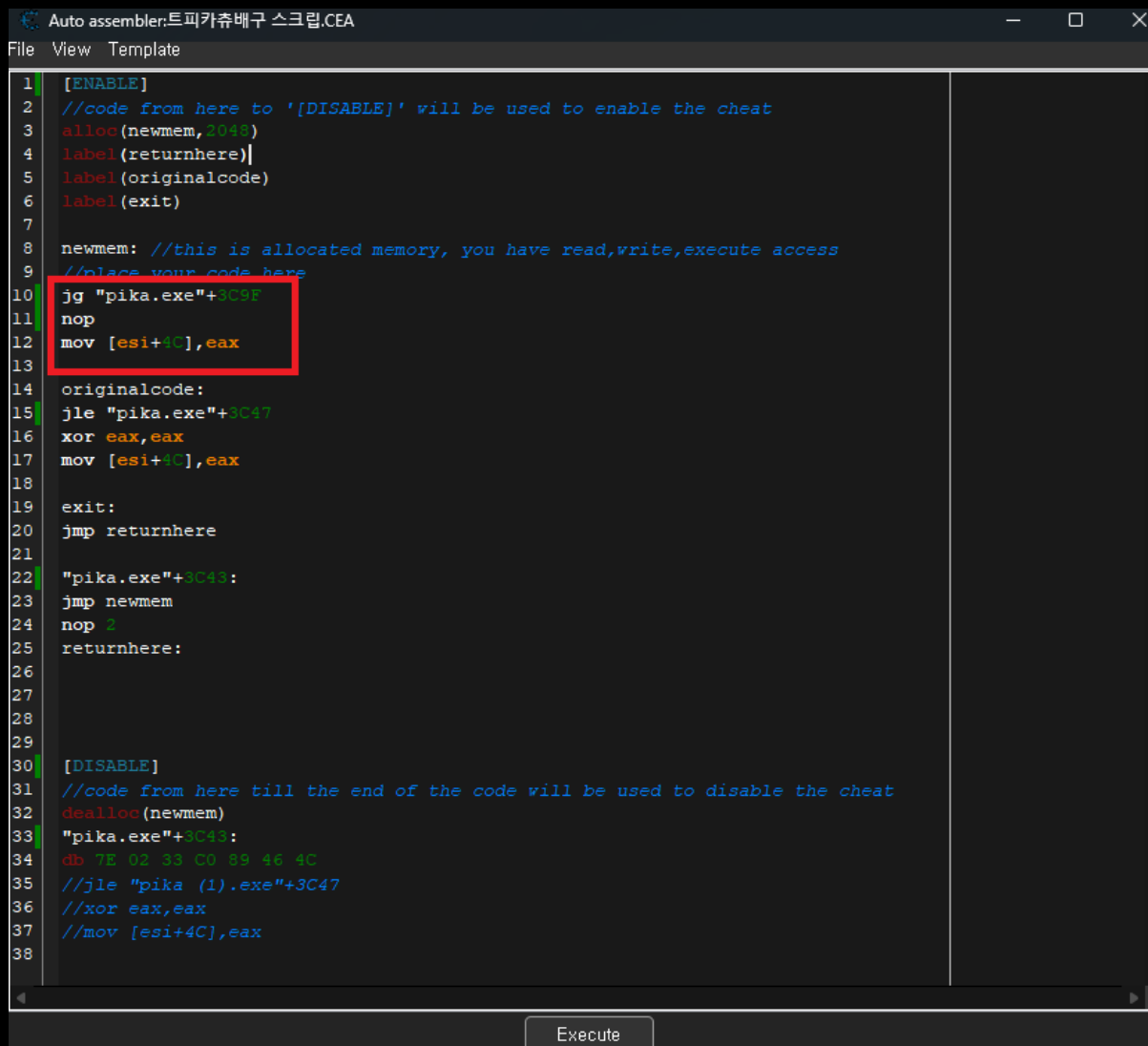
만약 jle라면..?

- 어떻게 됐을까.?

스크립트 작성하기



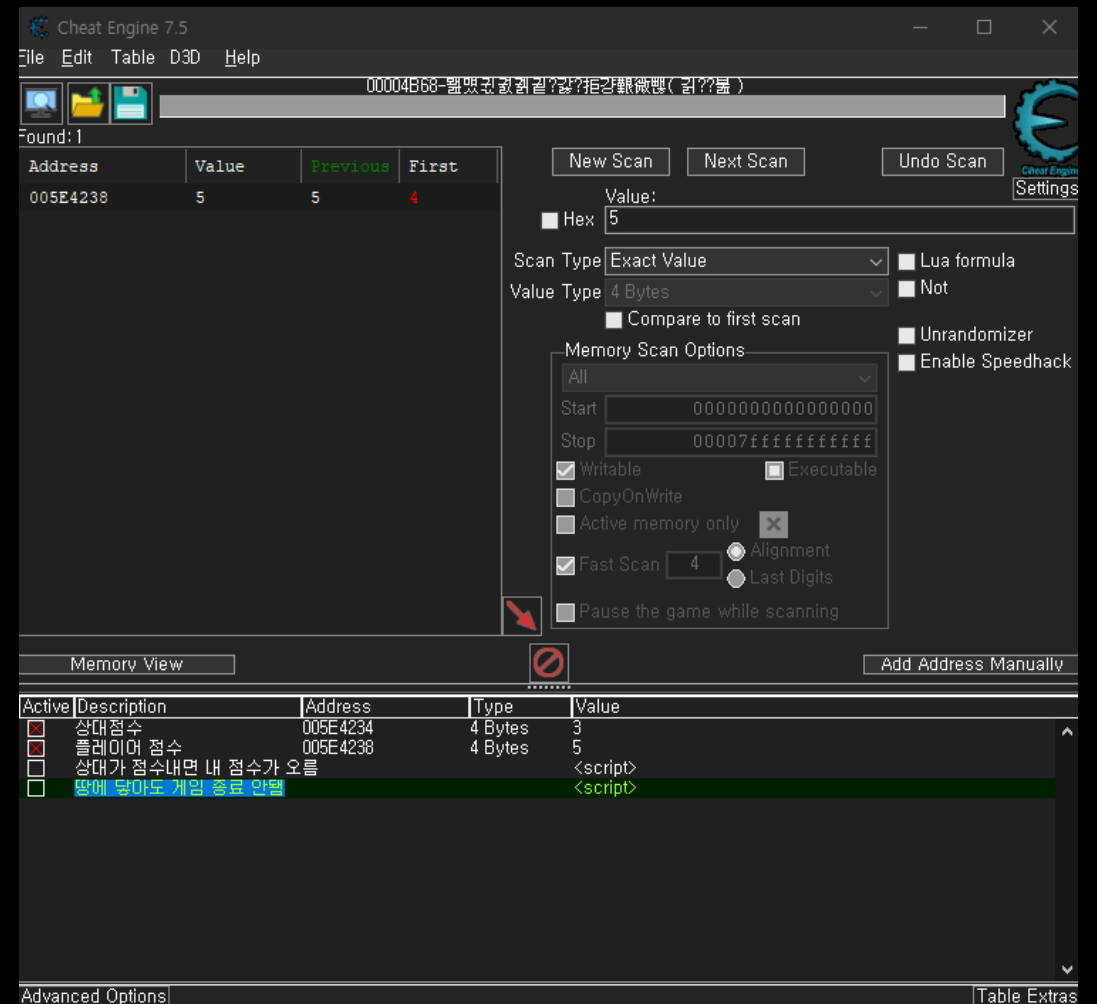
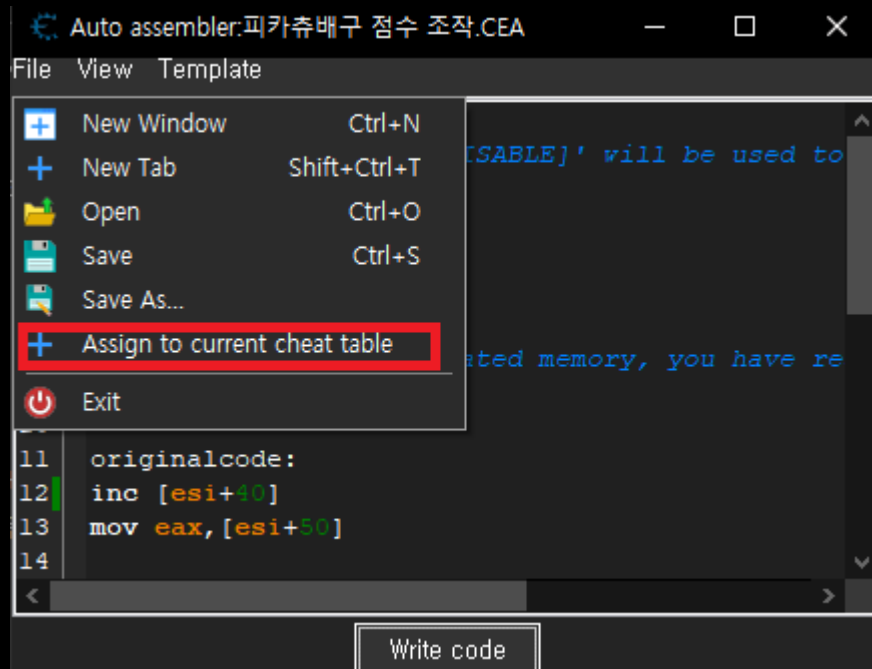
스크립트 작성하기 - 2



```
Auto assembler:트피카츄배구 스크립.CEA
File View Template
1 [ENABLE]
2 //code from here to '[DISABLE]' will be used to enable the cheat
3 alloc(newmem,2048)
4 label(returnhere)|
5 label(originalcode)
6 label(exit)
7
8 newmem: //this is allocated memory, you have read,write,execute access
9 //place your code here
10 jg "pika.exe"+3C9F
11 nop
12 mov [esi+4C],eax
13
14 originalcode:
15 jle "pika.exe"+3C47
16 xor eax,eax
17 mov [esi+4C],eax
18
19 exit:
20 jmp returnhere
21
22 "pika.exe"+3C43:
23 jmp newmem
24 nop 2
25 returnhere:
26
27
28
29
30 [DISABLE]
31 //code from here till the end of the code will be used to disable the cheat
32 dealloc(newmem)
33 "pika.exe"+3C43:
34 db 7E 02 33 C0 89 46 4C
35 //jle "pika (1).exe"+3C47
36 //xor eax,eax
37 //mov [esi+4C],eax
38
```

Execute

작성한 스크립트



정리

- 피카츄 배구 핵을 만들었다.
- 이걸 어떻게 응용하고 적용하는지는 여러분 자유입니다.

다음?

- AssultCube 핵을 제작해보자.(실습)